Smart Medication Dispenser

Complete Software Documentation

SOFTWARE ENGINEERING REFERENCE

Designed and Engineered in Lausanne, Switzerland

Version: 1.1

Date: February 2026

Classification: Confidential - Engineering Team

Covers: Architecture, Backend API, Database, Cloud/Deployment, Web Portal, Mobile App, Authentication, Integrations

Table of Contents

01. Software Architecture

Clean Architecture, CQRS pattern, project layers, dependency flow

02. Backend API Reference

ASP.NET Core 8 — controllers, handlers, middleware, 40+ endpoints

03. Database Documentation

Entity Framework Core, schema design, migrations, relationships, seed data

04. Cloud & Deployment

Docker, PostgreSQL, environment configuration, CI/CD, production setup

05. Web Portal

React + Vite caregiver portal — routing, pages, components, state management

06. Mobile App

React Native / Expo patient app — navigation, screens, notifications

07. Authentication & Authorization

JWT auth, API keys, user roles, password hashing, security

08. Integrations & Webhooks

Outgoing/incoming webhooks, cloud sync, device API keys, external systems

Software Architecture

Smart Medication Dispenser — Clean Architecture & CQRS Reference

---

1. Architecture Overview

The backend follows **Clean Architecture** (also known as Onion Architecture) with the **CQRS** (Command Query Responsibility Segregation) pattern implemented via **MediatR**. This ensures:

- **Separation of concerns** — each layer has a single responsibility
- **Testability** — business logic is isolated from infrastructure
- **Maintainability** — changes in one layer don't cascade through others
- **Flexibility** — database, UI, and external services can be swapped independently

1.1 Layer Dependency Flow

```
┌──────────────────────────────────────────────────────────────────┐
│                     DEPENDENCY DIRECTION                         │
│                     (outer depends on inner)                     │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────────────────────────────────────────────┐       │
│   │  API Layer (Presentation)                            │       │
│   │  Controllers • Middleware • Program.cs               │       │
│   │  Depends on: Application, Infrastructure             │       │
│   │                                                      │       │
│   │   ┌──────────────────────────────────────────────┐   │       │
│   │   │  Infrastructure Layer                        │   │       │
│   │   │  EF Core • Repositories • JWT • Background Jobs │   │       │
│   │   │  Depends on: Application, Domain             │   │       │
│   │   │                                              │   │       │
│   │   │   ┌──────────────────────────────────┐       │   │       │
│   │   │   │  Application Layer               │       │   │       │
│   │   │   │  Commands • Queries • Handlers • DTOs    │   │       │
│   │   │   │  Depends on: Domain              │       │   │       │
│   │   │   │                                  │       │   │       │
│   │   │   │   ┌──────────────────────────┐   │       │   │       │
│   │   │   │   │  Domain Layer (Core)     │   │       │   │       │
│   │   │   │   │  Entities • Enums        │   │       │   │       │
│   │   │   │   │  NO external dependencies │   │       │   │       │
│   │   │   │   └──────────────────────────┘   │       │   │       │
│   │   │   │                                  │       │   │       │
│   │   │   └──────────────────────────────────┘       │   │       │
│   │   │                                              │   │       │
│   │   └──────────────────────────────────────────────┘   │       │
│   │                                                      │       │
│   └──────────────────────────────────────────────────────┘       │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

1.2 Key Design Rules

Rule	Description
Domain has no dependencies	Entities and enums depend on nothing external
Application defines interfaces	Repository interfaces live in Application, implementations in Infrastructure
Infrastructure implements	EF Core, JWT, webhooks, background jobs
API orchestrates	Configures DI container, middleware pipeline, routes requests to MediatR
No direct DB access from controllers	All data access goes through MediatR → Handler → Repository

---

2. Project Structure

2.1 Solution Layout

```
backend/
├── SmartMedicationDispenser.sln          # Visual Studio solution
└── src/
    ├── Api/                               # ASP.NET Core Web API
    │   ├── Api.csproj                     # References: Application, Infrastructure
    │   ├── Program.cs                     # Application bootstrap & DI configuration
    │   ├── Dockerfile                     # Container image definition
    │   ├── appsettings.json               # Configuration (DB, JWT)
    │   ├── appsettings.Development.json   # Dev overrides
    │   ├── Controllers/                   # 11 REST API controllers
    │   ├── Middleware/                     # Global exception handler
    │   └── Properties/
    │       └── launchSettings.json        # Dev server settings
    │
    ├── Application/                       # Business logic (CQRS)
    │   ├── Application.csproj             # References: Domain
    │   ├── DependencyInjection.cs         # MediatR + FluentValidation registration
    │   ├── Auth/                          # Register, Login, Me commands/queries
    │   ├── Devices/                       # Device CRUD + heartbeat
    │   ├── Containers/                    # Container CRUD
    │   ├── Schedules/                     # Schedule CRUD + today's schedule
    │   ├── Dispensing/                    # Dispense, confirm, delay
    │   ├── Notifications/                 # Notification list + mark read
    │   ├── Travel/                        # Start/end travel session
    │   ├── Integrations/                  # Webhooks, API keys, sync
    │   ├── Adherence/                     # Adherence statistics
    │   ├── Common/                        # Shared interfaces (repositories, services)
    │   ├── DTOs/                          # Data Transfer Objects
    │   └── Validators/                    # FluentValidation request validators
    │
    ├── Domain/                            # Core domain model
    │   ├── Domain.csproj                  # No project references
    │   ├── Entities/                      # 10 entity classes
    │   └── Enums/                         # 6 enums + 1 static constants class
    │
    └── Infrastructure/                    # External concerns
        ├── Infrastructure.csproj          # References: Application, Domain
        ├── DependencyInjection.cs         # Repository + service registration
        ├── Background/                    # Hosted services (missed dose checker)
        ├── Migrations/                    # EF Core migrations
        ├── Persistence/                   # DbContext, repositories, seed data
        └── Services/                      # JWT auth, webhook delivery, etc.
```

2.2 Project References

```
Api.csproj
 ├── Application.csproj
 │    └── Domain.csproj
 └── Infrastructure.csproj
      ├── Application.csproj
      │    └── Domain.csproj
      └── Domain.csproj
```

2.3 NuGet Package Versions

Package	Version	Project	Purpose
Microsoft.EntityFrameworkCore	8.0.10	Api, Infrastructure	ORM framework
Microsoft.EntityFrameworkCore.Design	8.0.10	Api, Infrastructure	EF CLI tooling
Microsoft.EntityFrameworkCore.Sqlite	8.0.10	Infrastructure	SQLite provider (dev)
Npgsql.EntityFrameworkCore.PostgreSQL	8.0.10	Infrastructure	PostgreSQL provider (prod)
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.10	Infrastructure	JWT auth middleware
BCrypt.Net-Next	4.0.3	Infrastructure	Password hashing
MediatR	12.2.0	Api, Application	CQRS mediator
FluentValidation	11.9.0	Api, Application	Request validation
FluentValidation.DependencyInjectionExtensions	11.9.0	Application	Auto-register validators
Swashbuckle.AspNetCore	6.6.2	Api	Swagger/OpenAPI
Microsoft.AspNetCore.OpenApi	8.0.10	Api	OpenAPI metadata

---

3. Domain Layer

The Domain layer is the **innermost** layer with **zero external dependencies**. It contains only pure C# classes and enums.

3.1 Entities (10 total)

Entity	Description	Key Properties
User	Application user	Id, Email, PasswordHash, FullName, Role, CaregiverUserId
Device	Physical dispenser	Id, UserId, Name, Type, Status, LastHeartbeatAtUtc, BatteryLevel
Container	Medication slot	Id, DeviceId, SlotNumber, MedicationName, Quantity, PillsPerDose
Schedule	Recurring dose time	Id, ContainerId, TimeOfDay, DaysOfWeekBitmask, StartDate, EndDate
DispenseEvent	Dose lifecycle	Id, DeviceId, ContainerId, ScheduleId, Status, ScheduledAtUtc
Notification	In-app alert	Id, UserId, Type, Title, Body, IsRead
TravelSession	Travel mode link	Id, UserId, MainDeviceId, PortableDeviceId, PlannedEndDateUtc
WebhookEndpoint	Outgoing webhook	Id, UserId, Url, Secret, IsActive
DeviceApiKey	Machine auth key	Id, DeviceId, KeyHash, Name

Entity	Description	Key Properties
DeviceEventLog	Raw device telemetry	Id, DeviceId, EventType, DataJson, Processed

3.2 Enums & Constants (6 enums + 1 static class)

Type	Kind	Values	Usage
UserRole	Enum	Patient (0), Caregiver (1), Admin (2)	User access level
DeviceType	Enum	Main (0), Portable (1)	SMD-100 vs SMD-200
DeviceStatus	Enum	Active (0), Paused (1)	Dispensing state
DispenseEventStatus	Enum	Pending (0), Dispensed (1), Confirmed (2), Missed (3), Delayed (4)	Dose lifecycle
NotificationType	Enum	MissedDose, LowStock, TravelStarted, TravelEnded, General, DoseDispensed, DoseTaken, RefillCritical, DeviceOnline, DeviceOffline, DeviceError, DeviceStatus, BatteryLow, BatteryCritical	Notification category
DeviceEventType	Enum	DoseDispensed, DoseTaken, DoseMissed, RefillNeeded, RefillCritical, DeviceOnline, DeviceOffline, DeviceError, BatteryLow, BatteryCritical, TravelModeOn, TravelModeOff, Heartbeat	Device telemetry
DeviceErrorCode	Static class	E001-E099 (Network), E101-E199 (Hardware), E201-E299 (Power), E301-E399 (Storage)	Error code constants (not an enum — uses string constants like `"E101"`)

3.3 Entity Relationships

```
User ──┬── 1:N ──▶ Device ──┬── 1:N ──▶ Container ──┬── 1:N ──▶ Schedule
       │                    │                       │            │
       │                    │                       │            └── 1:N ──▶ DispenseEvent
       │                    │                       │
       │                    │                       └── 1:N ──▶ DispenseEvent
       │                    │
       │                    ├── 1:N ──▶ DispenseEvent
       │                    │
       │                    ├── 1:N ──▶ DeviceApiKey
       │                    │
       │                    └── 1:N ──▶ DeviceEventLog
       │
       ├── 1:N ──▶ Notification
       │
       ├── 1:N ──▶ WebhookEndpoint
       │
       └── Self-referential (CaregiverUserId → User)
             Caregiver ── 1:N ──▶ Patients

TravelSession links:
  User ──▶ TravelSession ◀── MainDevice
                         ◀── PortableDevice

Container self-referential:
  SourceContainer ── 1:N ──▶ PortableCopies
```

---

4. Application Layer

The Application layer contains all **business logic** organized as CQRS commands and queries dispatched via MediatR.

4.1 CQRS Pattern

```
┌──────────────────────────────────────────────────────────────┐
│                        CQRS FLOW                             │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Controller                                                  │
│      │                                                       │
│      ▼                                                       │
│  MediatR.Send(Command/Query)                                 │
│      │                                                       │
│      ├──▶ Command → CommandHandler → Repository → DB (write)  │
│      │       Returns: DTO or void                            │
│      │                                                       │
│      └──▶ Query → QueryHandler → Repository → DB (read)      │
│             Returns: DTO or list of DTOs                     │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

4.2 Command/Query Inventory

Module	Commands	Queries	Total Handlers
Auth	RegisterCommand, LoginCommand	MeQuery, MeProfileQuery	4
Devices	CreateDeviceCommand, PauseDeviceCommand, ResumeDeviceCommand, HeartbeatCommand	GetDevicesQuery, GetDeviceByIdQuery	6
Containers	CreateContainerCommand, UpdateContainerCommand, DeleteContainerCommand	GetContainersByDeviceQuery	4
Schedules	CreateScheduleCommand, UpdateScheduleCommand, DeleteScheduleCommand	GetSchedulesByContainerQuery, GetTodayScheduleQuery	5
Dispensing	DispenseCommand, ConfirmDispenseCommand, DelayDispenseCommand	GetDeviceEventsQuery	4
Notifications	MarkNotificationReadCommand	GetNotificationsQuery	2
Travel	StartTravelCommand, EndTravelCommand	—	2
Integrations	CreateWebhookEndpointCommand, DeleteWebhookEndpointCommand, CreateDeviceApiKeyCommand, DeleteDeviceApiKeyCommand, ProcessIncomingWebhookCommand, SyncFromCloudCommand	GetWebhookEndpointsQuery, GetDeviceApiKeysQuery	8
Adherence	—	GetAdherenceQuery	1
Total	**18 Commands**	**10 Queries**	**36 Handlers**

4.3 Dependency Injection Registration

```
// Application/DependencyInjection.cs
public static IServiceCollection AddApplication(this IServiceCollection services)
{
    // Auto-register all MediatR handlers from this assembly
    services.AddMediatR(cfg => cfg.RegisterServicesFromAssembly(Assembly.GetExecutingAssembly()));

    // Auto-register all FluentValidation validators
    services.AddValidatorsFromAssemblyContaining<RegisterRequestValidator>();

    return services;
}
```

4.4 Common Interfaces

The Application layer defines **interfaces** for all external dependencies. These are implemented in the Infrastructure layer:

Interface	Purpose	Implementation
`IUserRepository`	User CRUD	`UserRepository`
`IDeviceRepository`	Device CRUD + heartbeat	`DeviceRepository`
`IContainerRepository`	Container CRUD + low stock	`ContainerRepository`
`IScheduleRepository`	Schedule CRUD + today	`ScheduleRepository`
`IDispenseEventRepository`	Dispense event lifecycle	`DispenseEventRepository`
`ITravelSessionRepository`	Travel session management	`TravelSessionRepository`
`INotificationRepository`	Notification CRUD	`NotificationRepository`
`IWebhookEndpointRepository`	Webhook endpoint CRUD	`WebhookEndpointRepository`
`IDeviceApiKeyRepository`	API key management	`DeviceApiKeyRepository`
`IUnitOfWork`	Transaction management	`UnitOfWork`
`IAuthService`	JWT generation + password hashing	`JwtAuthService`
`IWebhookDeliveryService`	HTTP POST to webhook URLs	`WebhookDeliveryService`
`IDeviceApiKeyResolver`	Resolve device from API key	`DeviceApiKeyResolver`
`IDateTimeProvider`	Mockable UTC clock	`DateTimeProvider`

4.5 Validators

FluentValidation validators ensure request data integrity before handlers execute:

Validator	Rules
`RegisterRequestValidator`	Email required & valid format, password min length, fullName required, role valid
`LoginRequestValidator`	Email required, password required
`CreateDeviceRequestValidator`	Name required & max length, type valid (Main/Portable)
`CreateContainerRequestValidator`	SlotNumber > 0, MedicationName required, Quantity ≥ 0, PillsPerDose > 0
`CreateScheduleRequestValidator`	TimeOfDay valid, DaysOfWeekBitmask 1-127, StartDate required

4.6 DTOs (Data Transfer Objects)

DTOs are defined in `Application/DTOs/` and serve as the **API contract** between frontend and backend:

DTO File	Types Defined
`AuthDtos.cs`	RegisterRequest, LoginRequest, AuthResponse, MeResponse, MeProfileResponse, UserDeviceSummaryDto
`DeviceDtos.cs`	DeviceDto, DeviceDetailDto, ContainerSlotDto, CreateDeviceRequest, UpdateDeviceRequest, DeviceHealthDto
`ContainerDtos.cs`	CreateContainerRequest, UpdateContainerRequest, ContainerDto
`ScheduleDtos.cs`	CreateScheduleRequest, UpdateScheduleRequest, ScheduleDto, TodayScheduleItemDto
`DispenseDtos.cs`	DispenseRequest, ConfirmDispenseRequest, DelayDispenseRequest, DispenseEventDto
`NotificationDtos.cs`	NotificationDto
`AdherenceDtos.cs`	AdherenceSummaryDto
`TravelDtos.cs`	StartTravelRequest, TravelSessionDto
`WebhookDtos.cs`	CreateWebhookEndpointRequest, UpdateWebhookEndpointRequest, WebhookEndpointDto, IncomingWebhookPayload, SyncRequest, SyncDeviceStatus, SyncDispenseEventDto, SyncContainerDto
`DeviceApiKeyDtos.cs`	DeviceApiKeyDto
`DeviceEventDtos.cs`	DeviceEventPayload, DoseDispensedData, DoseTakenData, DoseMissedData, RefillNeededData, DeviceOnlineData, DeviceOfflineData, DeviceErrorData, BatteryLowData, TravelModeOnData, TravelModeOffData, HeartbeatPayload, HeartbeatSlot, DeviceRegistrationRequest/Response, ScheduleItem, DeviceScheduleResponse, FirmwareUpdateResponse

---

5. Infrastructure Layer

The Infrastructure layer implements all external concerns: database access, authentication, background jobs, and webhook delivery.

5.1 Dependency Injection Registration

```
// Infrastructure/DependencyInjection.cs
public static IServiceCollection AddInfrastructure(this IServiceCollection services, IConfiguratic
{
    // Database: auto-detect PostgreSQL vs SQLite from connection string
    var conn = configuration.GetConnectionString("DefaultConnection");
    if (conn.StartsWith("Data Source="))
        services.AddDbContext<AppDbContext>(o => o.UseSqlite(conn));
    else
        services.AddDbContext<AppDbContext>(o => o.UseNpgsql(conn));

    // Repositories
    services.AddScoped<IUnitOfWork, UnitOfWork>();
    services.AddScoped<IUserRepository, UserRepository>();
    services.AddScoped<IDeviceRepository, DeviceRepository>();
    // ... (9 repository registrations)

    // Services
    services.AddScoped<IAuthService, JwtAuthService>();
    services.AddScoped<IWebhookDeliveryService, WebhookDeliveryService>();
    services.AddScoped<IDeviceApiKeyResolver, DeviceApiKeyResolver>();
    services.AddSingleton<IDateTimeProvider, DateTimeProvider>();

    // Background jobs
    services.AddHostedService<MissedDoseAndLowStockHostedService>();

    return services;
}
```

5.2 Background Services

Service	Interval	Purpose
`MissedDoseAndLowStockHostedService`	Every 5 minutes	Marks overdue dispense events as Missed (60 min threshold), creates LowStock notifications, sends webhooks to patient and caregiver

5.3 External Service Implementations

Service	Description
`JwtAuthService`	Generates HS256 JWT tokens (7-day expiry), hashes passwords with BCrypt
`WebhookDeliveryService`	POSTs JSON payloads to webhook URLs with optional HMAC-SHA256 signature
`DeviceApiKeyResolver`	Resolves device ID from X-API-Key header by SHA256 hash lookup
`DateTimeProvider`	Wraps `DateTime.UtcNow` for testability

---

6. API Layer (Presentation)

The API layer is the entry point. It configures the ASP.NET Core pipeline and routes HTTP requests to MediatR handlers.

6.1 Startup Pipeline (Program.cs)

```
1. AddControllers()              — Register MVC controllers
2. AddSwaggerGen()               — OpenAPI documentation
3. AddAuthentication(JWT Bearer) — Validate JWT tokens
4. AddApplication()              — MediatR + FluentValidation
5. AddInfrastructure()           — EF Core, repositories, services
6. UseMiddleware<GlobalException> — Catch unhandled exceptions → JSON
7. UseSwagger() (dev only)       — Swagger UI
8. UseCors(AllowAny)             — Cross-origin requests (MVP)
9. UseAuthentication()           — JWT validation middleware
10. UseAuthorization()           — Role-based access
11. MapControllers()             — Route to controllers
12. Migrate + Seed               — Auto-apply migrations and seed data
```

6.2 Controller Summary (11 Controllers)

Controller	Route	Auth	Endpoints
`AuthController`	`/api/auth`	None / JWT	Register, Login, Me, Me/Profile
`DevicesController`	`/api/devices`	JWT	List, Get, Create, Pause, Resume, Heartbeat
`ContainersController`	`/api/containers`	JWT	List by device, Create, Update, Delete
`SchedulesController`	`/api/schedules`	JWT	List by container, Create, Update, Delete, Today
`DispensingController`	`/api`	JWT	Dispense, Confirm, Delay
`HistoryController`	`/api`	JWT	Get device events (date filter)
`PatientsController`	`/api/patients`	JWT	Adherence summary
`NotificationsController`	`/api/notifications`	JWT	List, Mark as read
`TravelController`	`/api/travel`	JWT	Start, End
`IntegrationsController`	`/api/integrations`	JWT / X-API-Key	Webhooks CRUD, API keys CRUD, Sync
`DeviceApiController`	`/api/v1`	Device Token / X-API-Key	Register, Heartbeat, Schedule, Events, Inventory, Error, Firmware
`WebhooksController`	`/api/webhooks`	X-API-Key	Incoming webhook

6.3 Middleware

Middleware	Purpose
`GlobalExceptionMiddleware`	Catches all unhandled exceptions; maps to appropriate HTTP status codes; returns consistent JSON error body

Exception mapping:

Exception Type	HTTP Status	Response
`ValidationException`	400 Bad Request	Validation error messages
`UnauthorizedAccessException`	401 Unauthorized	"Unauthorized."
`KeyNotFoundException`	404 Not Found	Exception message
`ArgumentException`	400 Bad Request	Exception message
All others	500 Internal Server Error	Generic message (or detail in dev)

---

7. Request Flow Example

Example: Confirming a Dose Intake

```
1. Mobile App: POST /api/dispense-events/{id}/confirm
   Headers: Authorization: Bearer <jwt>

2. ASP.NET Core Pipeline:
   → GlobalExceptionMiddleware (wraps entire request)
   → JWT Bearer Authentication (validates token, sets ClaimsPrincipal)
   → Authorization (checks [Authorize] attribute)
   → Routing → DispensingController.Confirm()

3. Controller:
   → Extracts userId from JWT claims
   → Sends ConfirmDispenseCommand(userId, eventId) via MediatR

4. ConfirmDispenseCommandHandler:
   → Loads DispenseEvent from IDispenseEventRepository
   → Validates: event exists, belongs to user, status is Dispensed/Pending
   → Updates: Status = Confirmed, ConfirmedAtUtc = now
   → Decrements Container.Quantity by PillsPerDose
   → Triggers outgoing webhooks (dispense.confirmed)
   → Saves via IUnitOfWork.SaveChangesAsync()
   → Returns DispenseEventDto

5. Response: 200 OK with DispenseEventDto JSON
```

---

8. Testing Architecture

8.1 Test Projects

Project	Type	Framework	Covers
`Application.Tests`	Unit	xUnit	Command/query handlers, business logic
`Domain.Tests`	Unit	xUnit	Entity behavior, enum validation

8.2 Test Categories

Category	Example Tests
Handler tests	ConfirmDispenseCommandHandlerTests — verifies dose confirmation flow
Missed dose logic	MissedDoseLogicTests — verifies 60-minute timeout marking
Travel mode	TravelModeLogicTests — verifies container copying and session management
Domain	DomainTests — entity creation, property validation

8.3 Running Tests

```
cd backend
dotnet test                              # Run all tests
dotnet test --filter "Category=Unit"     # Run unit tests only
dotnet test --verbosity detailed         # Verbose output
```

---

9. Design Patterns Used

Pattern	Where Used	Purpose
Clean Architecture	Entire backend	Layer separation, dependency inversion
CQRS	Application layer	Separate read/write models
Mediator	MediatR dispatch	Decouple controllers from handlers
Repository	Infrastructure layer	Abstract database access
Unit of Work	`UnitOfWork` class	Transactional consistency
DTO	Application/DTOs	API contract separation from domain
Strategy	`GlobalExceptionMiddleware`	Exception-to-HTTP-status mapping
Observer	Webhooks	External event notification
Background Service	`MissedDoseAndLowStockHostedService`	Async processing

Pattern	Where Used	Purpose
Factory	`JwtAuthService.GenerateJwt()`	Token creation

Backend API Reference

Smart Medication Dispenser — ASP.NET Core 8 Web API

---

1. Overview

The backend is an **ASP.NET Core 8 Web API** built with Clean Architecture. It exposes two API layers:

API Layer	Base Path	Auth Method	Consumers
User/App API	`/api/*`	JWT Bearer (email/password login)	Web portal, Mobile app
Device API	`/api/v1/*`	Device JWT or X-API-Key header	ESP32 firmware

Base URLs

Environment	URL
Development	`http://localhost:5000`
Docker	`http://localhost:5000`
Swagger UI	`http://localhost:5000/swagger`

---

2. User/App API — Complete Endpoint Reference

2.1 Authentication (`AuthController`)

Method	Endpoint	Auth	Description
POST	`/api/auth/register`	None	Register a new user
POST	`/api/auth/login`	None	Login with email/password
GET	`/api/auth/me`	JWT	Get current user (id, email, name, role)
GET	`/api/auth/me/profile`	JWT	Get profile with owned device list

POST `/api/auth/register`

Request:

```
{
  "email": "user@example.com",
  "password": "SecurePass123!",
  "fullName": "John Doe",
  "role": "Patient"
}
```

Response (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "email": "user@example.com",
  "fullName": "John Doe",
  "role": "Patient",
  "userId": "a1b2c3d4-..."
}
```

Errors: `400` — Email already exists or validation failed.

POST `/api/auth/login`

Request:

```
{
  "email": "patient@demo.com",
  "password": "Demo123!"
}
```

Response (200): Same as register response (AuthResponse).

Errors: `401` — Invalid email or password.

GET `/api/auth/me`

Headers: `Authorization: Bearer <jwt>`

Response (200):

```
{
  "id": "a1b2c3d4-...",
  "email": "patient@demo.com",
  "fullName": "Demo Patient",
  "role": "Patient"
}
```

GET `/api/auth/me/profile`

Headers: `Authorization: Bearer <jwt>`

Response (200):

```
{
  "userId": "a1b2c3d4-...",
  "email": "patient@demo.com",
  "fullName": "Demo Patient",
  "role": "Patient",
  "devices": [
    { "deviceId": "d1e2f3...", "name": "Home Dispenser", "type": "Main", "status": "Active" },
    { "deviceId": "d4e5f6...", "name": "Travel Dispenser", "type": "Portable", "status": "Paused"
  ]
}
```

2.2 Devices (`DevicesController`)

All endpoints require JWT Bearer authentication. User can only access their own devices.

Method	Endpoint	Description
GET	`/api/devices`	List all user's devices
GET	`/api/devices/{id}`	Get single device
POST	`/api/devices`	Create a new device
PATCH	`/api/devices/{id}/pause`	Pause dispensing
PATCH	`/api/devices/{id}/resume`	Resume dispensing
POST	`/api/devices/{id}/heartbeat`	Record device heartbeat

POST `/api/devices`

Request:

```
{
  "name": "Home Dispenser",
  "type": "Main",
  "timeZoneId": "Europe/Zurich"
}
```

Response (201) — DeviceDto:

```
{
  "id": "d1e2f3...",
  "userId": "a1b2c3...",
  "name": "Home Dispenser",
  "type": "Main",
  "status": "Active",
  "timeZoneId": "Europe/Zurich",
  "lastHeartbeatAtUtc": null,
  "firmwareVersion": null,
  "isOnline": false,
  "batteryLevel": null,
  "wifiSignal": null,
  "temperature": null,
  "humidity": null
}
```

GET `/api/devices/{id}`

Response (200) — DeviceDetailDto: Returns full device details including hardware info and container slots:

```
{
  "id": "d1e2f3...",
  "userId": "a1b2c3...",
  "name": "Home Dispenser",
  "type": "Main",
  "status": "Active",
  "timeZoneId": "Europe/Zurich",
  "lastHeartbeatAtUtc": "2026-02-10T08:30:00Z",
  "firmwareVersion": "2.1.0",
  "hardwareVersion": "1.0",
  "macAddress": "AA:BB:CC:DD:EE:FF",
  "isOnline": true,
  "batteryLevel": 85,
  "wifiSignal": -45,
  "temperature": 22.5,
  "humidity": 40,
  "lastOnlineAtUtc": "2026-02-10T08:30:00Z",
  "lastOfflineAtUtc": null,
  "lastErrorCode": null,
  "lastErrorMessage": null,
  "createdAtUtc": "2026-01-15T10:00:00Z",
  "containers": [
    {
      "slotNumber": 1,
      "medicationName": "Metformin 500mg",
      "pillsRemaining": 85,
      "capacity": 90,
      "lowStockThreshold": 10,
      "status": "ok"
    }
  ]
}
```

PATCH `/api/devices/{id}/pause`

Response (200): DeviceDto with `status: "Paused"`

PATCH `/api/devices/{id}/resume`

Response (200): DeviceDto with `status: "Active"`

POST `/api/devices/{id}/heartbeat`

Response: `204 No Content`

2.3 Containers (`ContainersController`)

Method	Endpoint	Description
GET	`/api/devices/{deviceId}/containers`	List containers for device
POST	`/api/devices/{deviceId}/containers`	Create a container
PUT	`/api/containers/{id}`	Update a container
DELETE	`/api/containers/{id}`	Delete a container

POST `/api/devices/{deviceId}/containers`

Request:

```
{
  "slotNumber": 1,
  "medicationName": "Metformin 500mg",
  "medicationImageUrl": null,
  "quantity": 90,
  "pillsPerDose": 2,
  "lowStockThreshold": 14,
  "sourceContainerId": null
}
```

Response (201):

```
{
  "id": "c1d2e3...",
  "deviceId": "d1e2f3...",
  "slotNumber": 1,
  "medicationName": "Metformin 500mg",
  "medicationImageUrl": null,
  "quantity": 90,
  "pillsPerDose": 2,
  "lowStockThreshold": 14,
  "sourceContainerId": null
}
```

PUT `/api/containers/{id}`

Request:

```
{
  "slotNumber": 1,
  "medicationName": "Metformin 500mg",
  "medicationImageUrl": null,
  "quantity": 85,
  "pillsPerDose": 2,
  "lowStockThreshold": 14
}
```

2.4 Schedules (`SchedulesController`)

Method	Endpoint	Description
GET	`/api/containers/{containerId}/schedules`	List schedules for container
POST	`/api/containers/{containerId}/schedules`	Create a schedule
PUT	`/api/schedules/{id}`	Update a schedule
DELETE	`/api/schedules/{id}`	Delete a schedule
GET	`/api/devices/{deviceId}/today-schedule`	Get today's scheduled doses

POST `/api/containers/{containerId}/schedules`

Request:

```
{
  "timeOfDay": "08:00",
  "daysOfWeekBitmask": 127,
  "startDate": "2026-02-01T00:00:00Z",
  "endDate": null,
  "notes": "Take with breakfast",
  "timeZoneId": "Europe/Zurich"
}
```

Days of Week Bitmask:

Bit	Day	Value
0	Sunday	1
1	Monday	2
2	Tuesday	4
3	Wednesday	8
4	Thursday	16
5	Friday	32
6	Saturday	64

Bit	Day	Value
All	Every day	127
Weekdays	Mon-Fri	62
Weekends	Sat-Sun	65

GET `/api/devices/{deviceId}/today-schedule?timeZoneId=Europe/Zurich`

Response (200):

```
[
  {
    "scheduleId": "s1a2b3...",
    "containerId": "c1d2e3...",
    "slotNumber": 1,
    "medicationName": "Metformin 500mg",
    "medicationImageUrl": null,
    "pillsPerDose": 2,
    "scheduledAtUtc": "2026-02-10T07:00:00Z",
    "notes": "Take with breakfast"
  }
]
```

2.5 Dispensing (`DispensingController`)

Method	Endpoint	Description
POST	`/api/devices/{deviceId}/dispense`	Trigger medication dispense
POST	`/api/dispense-events/{id}/confirm`	Confirm dose intake (taken)
POST	`/api/dispense-events/{id}/delay`	Delay reminder by N minutes

POST `/api/devices/{deviceId}/dispense`

Request:

```
{
  "scheduleId": "s1a2b3..."
}
```

Response (200):

```
{
  "id": "e1f2g3...",
  "deviceId": "d1e2f3...",
  "containerId": "c1d2e3...",
  "scheduleId": "s1a2b3...",
  "scheduledAtUtc": "2026-02-10T07:00:00Z",
  "status": "Dispensed",
  "dispensedAtUtc": "2026-02-10T07:00:05Z",
  "confirmedAtUtc": null,
  "missedMarkedAtUtc": null,
  "medicationName": "Metformin 500mg",
  "pillsPerDose": 2
}
```

POST `/api/dispense-events/{id}/confirm`

Response (200): DispenseEventDto with `status: "Confirmed"`, `confirmedAtUtc` set.

POST `/api/dispense-events/{id}/delay`

Request:

```
{
  "minutes": 15
}
```

Response (200): DispenseEventDto with `status: "Delayed"`.

2.6 History (`HistoryController`)

Method	Endpoint	Description
GET	`/api/devices/{deviceId}/events`	Get dispense event history

Query Parameters:

Parameter	Type	Default	Description
`fromUtc`	DateTime?	null	Start date filter
`toUtc`	DateTime?	null	End date filter
`limit`	int	100	Max results

Response (200): Array of DispenseEventDto objects.

2.7 Adherence (`PatientsController`)

Method	Endpoint	Description
GET	`/api/patients/me/adherence`	Get adherence summary

Query Parameters: `fromUtc`, `toUtc` (optional date range).

Response (200):

```
{
  "totalScheduled": 28,
  "confirmed": 25,
  "missed": 2,
  "pending": 1,
  "adherencePercent": 89.3
}
```

2.8 Notifications (`NotificationsController`)

Method	Endpoint	Description
GET	`/api/notifications?limit=50`	List notifications
POST	`/api/notifications/{id}/read`	Mark notification as read

Response (200):

```
[
  {
    "id": "n1a2b3...",
    "type": "MissedDose",
    "title": "Missed dose",
    "body": "A scheduled dose was not confirmed in time.",
    "isRead": false,
    "createdAtUtc": "2026-02-10T08:05:00Z"
  }
]
```

2.9 Travel (`TravelController`)

Method	Endpoint	Description
POST	`/api/travel/start`	Start travel session
POST	`/api/travel/end`	End travel session

POST `/api/travel/start`

Request:

```
{
  "portableDeviceId": "d4e5f6...",
  "days": 7
}
```

Response (200):

```
{
  "id": "t1a2b3...",
  "mainDeviceId": "d1e2f3...",
  "portableDeviceId": "d4e5f6...",
  "startedAtUtc": "2026-02-10T10:00:00Z",
  "endedAtUtc": null,
  "plannedEndDateUtc": "2026-02-17T10:00:00Z"
}
```

Business Rules:
- Max 14 days per travel session
- Copies containers from main device to portable device
- Pauses main device, activates portable device
- Only one active travel session per user

POST `/api/travel/end`

Ends the current active travel session. Returns TravelSessionDto with `endedAtUtc` set.

2.10 Integrations (`IntegrationsController`)

Method	Endpoint	Auth	Description
GET	`/api/integrations/webhooks`	JWT	List outgoing webhooks
POST	`/api/integrations/webhooks`	JWT	Create webhook endpoint
DELETE	`/api/integrations/webhooks/{id}`	JWT	Delete webhook endpoint
GET	`/api/integrations/devices/{deviceId}/api-keys`	JWT	List API keys for device
POST	`/api/integrations/devices/{deviceId}/api-keys`	JWT	Create API key
DELETE	`/api/integrations/devices/{deviceId}/api-keys/{apiKeyId}`	JWT	Revoke API key

Method	Endpoint	Auth	Description
POST	`/api/integrations/sync`	X-API-Key	Sync data from cloud

See 08_INTEGRATIONS_WEBHOOKS.md for detailed integration documentation.

---

3. Device API — Complete Endpoint Reference

The Device API (`/api/v1/*`) is designed for ESP32 firmware communication with the cloud backend.

3.1 Endpoints

Method	Endpoint	Auth	Description
POST	`/api/v1/devices/register`	None	Register new device
POST	`/api/v1/devices/{deviceId}/heartbeat`	Device Token / X-API-Key	Send heartbeat
GET	`/api/v1/devices/{deviceId}/schedule`	Device Token / X-API-Key	Get medication schedule
POST	`/api/v1/events`	Device Token / X-API-Key	Send device event
POST	`/api/v1/devices/{deviceId}/inventory`	Device Token / X-API-Key	Sync inventory
POST	`/api/v1/devices/{deviceId}/error`	Device Token / X-API-Key	Report error
GET	`/api/v1/devices/{deviceId}/firmware`	Device Token / X-API-Key	Check firmware update

3.2 Device Registration

POST `/api/v1/devices/register`

```
{
  "deviceId": "SMD-100-ABCD1234",
  "deviceType": "SMD-100",
  "firmwareVersion": "1.0.0",
  "hardwareVersion": "2.0",
  "macAddress": "AA:BB:CC:DD:EE:FF"
}
```

Response:

```
{
  "success": true,
  "deviceToken": "device_token_...",
  "apiEndpoint": "https://api.smartdispenser.ch/api/v1",
  "heartbeatInterval": 60
}
```

3.3 Heartbeat

POST `/api/v1/devices/{deviceId}/heartbeat`

```
{
  "battery": 85,
  "wifiSignal": -45,
  "temperature": 22.5,
  "humidity": 45,
  "slots": [
    { "slot": 1, "pillCount": 87, "motorOk": true },
    { "slot": 2, "pillCount": 55, "motorOk": true }
  ],
  "errors": [],
  "freeMemory": 180000,
  "uptime": 86400
}
```

Response:

```
{
  "success": true,
  "server_time": "2026-02-10T12:00:00Z",
  "commands": [],
  "next_heartbeat": 60
}
```

3.4 Device Events

POST `/api/v1/events`

```
{
  "deviceId": "d1e2f3...",
  "event": "DOSE_DISPENSED",
  "timestamp": "2026-02-10T08:00:05Z",
  "data": {
    "slot": 1,
    "pillsDispensed": 2,
    "scheduleId": "s1a2b3..."
  }
}
```

Supported Event Types:

Event	Description	Priority
`DOSE_DISPENSED`	Pills released from slot	High
`DOSE_TAKEN`	Pills removed from tray (weight sensor)	High
`DOSE_MISSED`	Time window expired	Critical
`REFILL_NEEDED`	< 7 days of pills remaining	Medium
`REFILL_CRITICAL`	< 3 days of pills remaining	High
`DEVICE_ONLINE`	Connection established	Low
`DEVICE_OFFLINE`	Heartbeat timeout	Medium
`DEVICE_ERROR`	Hardware/software error	High
`BATTERY_LOW`	Battery < 20%	Medium
`BATTERY_CRITICAL`	Battery < 5%	High
`TRAVEL_MODE_ON`	Travel mode activated	Medium
`TRAVEL_MODE_OFF`	Travel mode deactivated	Medium

---

4. Error Handling

4.1 Global Exception Middleware

All unhandled exceptions are caught by `GlobalExceptionMiddleware` and returned as consistent JSON:

```
{
  "message": "Error description",
  "detail": "Error description"
}
```

In development mode, `stackTrace` is also included for 500 errors.

4.2 HTTP Status Code Reference

Status	Meaning	When Used
200	OK	Successful read or update
201	Created	Resource created (with Location header)
202	Accepted	Webhook/event accepted for processing
204	No Content	Successful delete or heartbeat
400	Bad Request	Validation failure or business rule violation

Status	Meaning	When Used
401	Unauthorized	Missing or invalid JWT / API key
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found or not owned by user
500	Internal Server Error	Unhandled exception

4.3 Validation Errors

FluentValidation errors return structured error messages:

```
{
  "message": "Email is required. Password must be at least 6 characters.",
  "detail": "Email is required. Password must be at least 6 characters."
}
```

---

5. Swagger / OpenAPI

Swagger UI is available in development at `http://localhost:5000/swagger`.

Features:

- **Try it out** — Execute API calls directly from the browser
- **Authorization** — Click "Authorize" and paste a JWT token to test protected endpoints
- **Schemas** — All request/response DTOs are documented
- **Endpoints** — All 40+ endpoints are listed with descriptions

Getting a JWT for Swagger:

1. Call `POST /api/auth/login` with demo credentials
2. Copy the `token` from the response
3. Click "Authorize" in Swagger UI
4. Enter `Bearer <token>` (include the "Bearer " prefix)
5. All subsequent requests will include the JWT

---

6. Configuration

6.1 appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5432;Database=dispenser;Username=dispenser;Password=
  },
  "Jwt": {
    "SecretKey": "SmartMedicationDispenser_MVP_SecretKey_AtLeast32Characters!",
    "Issuer": "SmartMedicationDispenser",
    "Audience": "SmartMedicationDispenser"
  }
}
```

6.2 Environment Variables

All settings can be overridden via environment variables:

Variable	Description	Default
`ConnectionStrings__DefaultConnection`	Database connection string	PostgreSQL localhost
`Jwt__SecretKey`	JWT signing key (min 32 chars)	Built-in dev key
`Jwt__Issuer`	JWT issuer claim	SmartMedicationDispenser
`Jwt__Audience`	JWT audience claim	SmartMedicationDispenser
`ASPNETCORE_ENVIRONMENT`	Runtime environment	Development
`ASPNETCORE_URLS`	Listen URLs	http://localhost:5000

6.3 Database Auto-Detection

The backend auto-detects the database provider from the connection string:

- **SQLite**: Connection string starts with `Data Source=` → uses `UseSqlite()`
- **PostgreSQL**: Any other format → uses `UseNpgsql()`

This allows seamless switching between local development (SQLite) and production (PostgreSQL).

Database Documentation

Smart Medication Dispenser — Entity Framework Core, Schema Design & Migrations

---

1. Overview

Aspect	Detail
ORM	Entity Framework Core 8.0
Production DB	PostgreSQL 15 (Alpine)
Development DB	SQLite (auto-detected from connection string)
Tables	10
Migrations	Code-first with EF Core Migrations
Auto-detect	Connection string starting with `Data Source=` → SQLite; otherwise → PostgreSQL

---

2. Database Schema

2.1 Entity-Relationship Diagram

```
┌──────────────────┐          ┌──────────────────┐
│      Users       │          │     Devices      │
├──────────────────┤          ├──────────────────┤
│ Id (PK, GUID)    │──1:N──▶│ Id (PK, GUID)    │
│ Email (unique)   │          │ UserId (FK)      │
│ PasswordHash     │          │ Name             │
│ FullName         │          │ Type (enum)      │
│ Role (enum)      │          │ Status (enum)    │
│ CaregiverUserId  │──self──▶│ TimeZoneId       │
│ CreatedAtUtc     │          │ LastHeartbeatAtUtc│
│ UpdatedAtUtc     │          │ FirmwareVersion  │
└──────────────────┘          │ HardwareVersion  │
        │                     │ MacAddress       │
        │                     │ BatteryLevel     │
        │                     │ WifiSignal       │
        │                     │ Temperature      │
        │                     │ Humidity         │
        │                     │ IsOnline         │
        │                     │ LastOnlineAtUtc  │
        │                     │ LastOfflineAtUtc │
        │                     │ LastErrorCode    │
        │                     │ LastErrorMessage │
        │                     │ CreatedAtUtc     │
        │                     │ UpdatedAtUtc     │
        │                     └──────────────────┘
        │                            │
        │                     ┌──────┼──────────────────────┐
        │                     │      │                      │
        │                     ▼      ▼                      ▼
        │          ┌──────────────┐ ┌──────────────┐  ┌──────────────────┐
        │          │  Containers  │ │DeviceApiKeys │  │ DeviceEventLogs  │
        │          ├──────────────┤ ├──────────────┤  ├──────────────────┤
        │          │ Id (PK)      │ │ Id (PK)      │  │ Id (PK)          │
        │          │ DeviceId(FK) │ │ DeviceId(FK) │  │ DeviceId (FK)    │
        │          │ SlotNumber   │ │ KeyHash      │  │ EventType (enum) │
        │          │ MedicationNm │ │ Name         │  │ EventTimestampUtc│
        │          │ MedImageUrl  │ │ CreatedAtUtc │  │ ReceivedAtUtc    │
        │          │ Quantity     │ │ LastUsedAtUtc│  │ DataJson         │
        │          │ PillsPerDose │ └──────────────┘  │ Processed        │
        │          │ LowStockThrs │                   │ ProcessingError  │
        │          │ SourceContId │                   └──────────────────┘
        │          │ CreatedAtUtc │
        │          │ UpdatedAtUtc │
        │          └──────────────┘
        │                 │
        │                 │ 1:N
        │                 ▼
        │          ┌──────────────┐
        │          │  Schedules   │
        │          ├──────────────┤
        │          │ Id (PK)      │
        │          │ ContainerId  │
        │          │ TimeOfDay    │
        │          │ DaysOfWeekBm │
        │          │ StartDate    │
```

```
        │         │ EndDate      │
        │         │ Notes        │
        │         │ TimeZoneId   │
        │         │ CreatedAtUtc │
        │         │ UpdatedAtUtc │
        │         └──────────────┘
        │               │
        │               │ 1:N
        │               ▼
        │         ┌──────────────────┐
        │         │  DispenseEvents   │
        │         ├──────────────────┤
        │         │ Id (PK)          │
        │         │ DeviceId (FK)    │
        │         │ ContainerId (FK) │
        │         │ ScheduleId (FK)  │
        │         │ ScheduledAtUtc   │
        │         │ Status (enum)    │
        │         │ DispensedAtUtc   │
        │         │ ConfirmedAtUtc   │
        │         │ MissedMarkedAtUtc│
        │         │ DelayedAtUtc     │
        │         │ CreatedAtUtc     │
        │         └──────────────────┘
        │
        ├──1:N──▶ ┌──────────────────┐
        │         │  Notifications    │
        │         ├──────────────────┤
        │         │ Id (PK)          │
        │         │ UserId (FK)      │
        │         │ Type (enum)      │
        │         │ Title            │
        │         │ Body             │
        │         │ IsRead           │
        │         │ CreatedAtUtc     │
        │         │ RelatedEntityId  │
        │         └──────────────────┘
        │
        └──1:N──▶ ┌──────────────────┐
                  │ WebhookEndpoints  │
                  ├──────────────────┤
                  │ Id (PK)          │
                  │ UserId (FK)      │
                  │ Url              │
                  │ Secret           │
                  │ IsActive         │
                  │ Description      │
                  │ LastTriggeredAt  │
                  │ LastStatus       │
                  │ CreatedAtUtc     │
                  └──────────────────┘


┌──────────────────────────┐
│    TravelSessions         │
├──────────────────────────┤
│ Id (PK)                  │
│ UserId (FK → Users)      │
│ MainDeviceId (FK)        │
│ PortableDeviceId (FK)    │
│ StartedAtUtc             │
│ EndedAtUtc               │
│ PlannedEndDateUtc        │
```

```
│ CreatedAtUtc                │
└─────────────────────────────┘
```

---

3. Table Definitions

3.1 Users

Column	Type	Constraints	Description
`Id`	GUID	PK	Unique user identifier
`Email`	VARCHAR(256)	Unique, Not Null	Login email
`PasswordHash`	TEXT	Not Null	BCrypt hash
`FullName`	VARCHAR(200)	Not Null	Display name
`Role`	INT (enum)	Not Null	0=Patient, 1=Caregiver, 2=Admin
`CaregiverUserId`	GUID?	FK → Users.Id	Assigned caregiver (self-ref)
`CreatedAtUtc`	TIMESTAMP	Not Null	Account creation time
`UpdatedAtUtc`	TIMESTAMP?	—	Last update time

Indexes: Unique index on `Email`

Relationships:
- Self-referential: `CaregiverUserId` → `Users.Id` (RESTRICT delete)
- One caregiver → many patients

3.2 Devices

Column	Type	Constraints	Description
`Id`	GUID	PK	Device identifier
`UserId`	GUID	FK → Users.Id, CASCADE	Owner
`Name`	VARCHAR(200)	Not Null	Device display name
`Type`	INT (enum)	Not Null	0=Main (SMD-100), 1=Portable (SMD-200)
`Status`	INT (enum)	Not Null	0=Active, 1=Paused
`TimeZoneId`	VARCHAR(100)	—	IANA time zone
`LastHeartbeatAtUtc`	TIMESTAMP?	—	Last heartbeat received
`FirmwareVersion`	VARCHAR(50)	—	Current firmware version

Column	Type	Constraints	Description
`HardwareVersion`	VARCHAR(50)	—	Hardware revision
`MacAddress`	VARCHAR(20)	—	WiFi MAC address
`BatteryLevel`	INT?	0-100	Battery percentage (portable)
`WifiSignal`	INT?	dBm	WiFi signal strength
`Temperature`	DECIMAL?	—	Ambient temperature (°C)
`Humidity`	INT?	0-100	Ambient humidity (%)
`IsOnline`	BOOLEAN	Not Null	Connection state
`LastOnlineAtUtc`	TIMESTAMP?	—	Last online timestamp
`LastOfflineAtUtc`	TIMESTAMP?	—	Last offline timestamp
`LastErrorCode`	VARCHAR(20)	—	Last error code
`LastErrorMessage`	VARCHAR(500)	—	Last error detail
`CreatedAtUtc`	TIMESTAMP	Not Null	Device creation time
`UpdatedAtUtc`	TIMESTAMP?	—	Last update time

Relationships:

- CASCADE delete from User → Devices
- One device → many containers, dispense events, API keys

3.3 Containers

Column	Type	Constraints	Description
`Id`	GUID	PK	Container identifier
`DeviceId`	GUID	FK → Devices.Id, CASCADE	Parent device
`SlotNumber`	INT	Not Null	Physical slot position
`MedicationName`	VARCHAR(200)	Not Null	Medication name
`MedicationImageUrl`	VARCHAR(500)	—	Image URL
`Quantity`	INT	Not Null	Current pill count
`PillsPerDose`	INT	Not Null	Pills dispensed per dose
`LowStockThreshold`	INT	Not Null	Alert threshold
`SourceContainerId`	GUID?	FK → Containers.Id, RESTRICT	Source for travel copies
`CreatedAtUtc`	TIMESTAMP	Not Null	Creation time
`UpdatedAtUtc`	TIMESTAMP?	—	Last update time

Relationships:
- CASCADE delete from Device → Containers
- Self-referential: `SourceContainerId` for travel container copies
- One container → many schedules, dispense events

3.4 Schedules

Column	Type	Constraints	Description
`Id`	GUID	PK	Schedule identifier
`ContainerId`	GUID	FK → Containers.Id, CASCADE	Parent container
`TimeOfDay`	TIME	Not Null	Dose time (e.g., 08:00)
`DaysOfWeekBitmask`	INT	Not Null	Active days (bitmask 1-127)
`StartDate`	TIMESTAMP	Not Null	Schedule start date
`EndDate`	TIMESTAMP?	—	Optional end date
`Notes`	VARCHAR(500)	—	Dosing instructions
`TimeZoneId`	VARCHAR(100)	—	IANA time zone
`CreatedAtUtc`	TIMESTAMP	Not Null	Creation time
`UpdatedAtUtc`	TIMESTAMP?	—	Last update time

Relationships:
- CASCADE delete from Container → Schedules
- One schedule → many dispense events

3.5 DispenseEvents

Column	Type	Constraints	Description
`Id`	GUID	PK	Event identifier
`DeviceId`	GUID	FK → Devices.Id, CASCADE	Source device
`ContainerId`	GUID	FK → Containers.Id, RESTRICT	Source container
`ScheduleId`	GUID	FK → Schedules.Id, RESTRICT	Source schedule
`ScheduledAtUtc`	TIMESTAMP	Not Null	When dose was scheduled
`Status`	INT (enum)	Not Null	0=Pending, 1=Dispensed, 2=Confirmed, 3=Missed, 4=Delayed
`DispensedAtUtc`	TIMESTAMP?	—	When pills were released
`ConfirmedAtUtc`	TIMESTAMP?	—	When user confirmed intake

Column	Type	Constraints	Description
`MissedMarkedAtUtc`	TIMESTAMP?	—	When marked as missed
`DelayedAtUtc`	TIMESTAMP?	—	When reminder was delayed
`CreatedAtUtc`	TIMESTAMP	Not Null	Record creation time

Status Lifecycle:

```
Pending → Dispensed → Confirmed (user took pills)
                    → Missed (60 min timeout)
                    → Delayed (user snoozed)
```

3.6 Notifications

Column	Type	Constraints	Description
`Id`	GUID	PK	Notification identifier
`UserId`	GUID	FK → Users.Id, CASCADE	Recipient
`Type`	INT (enum)	Not Null	Notification category
`Title`	VARCHAR(200)	Not Null	Notification title
`Body`	VARCHAR(2000)	Not Null	Notification body
`IsRead`	BOOLEAN	Not Null	Read status
`CreatedAtUtc`	TIMESTAMP	Not Null	Creation time
`RelatedEntityId`	GUID?	—	Related dispense event or container

3.7 TravelSessions

Column	Type	Constraints	Description
`Id`	GUID	PK	Session identifier
`UserId`	GUID	FK → Users.Id, CASCADE	Session owner
`MainDeviceId`	GUID	FK → Devices.Id, RESTRICT	Home device
`PortableDeviceId`	GUID	FK → Devices.Id, RESTRICT	Travel device
`StartedAtUtc`	TIMESTAMP	Not Null	Session start
`EndedAtUtc`	TIMESTAMP?	—	Session end (null if active)
`PlannedEndDateUtc`	TIMESTAMP	Not Null	Planned return date
`CreatedAtUtc`	TIMESTAMP	Not Null	Record creation time

3.8 WebhookEndpoints

Column	Type	Constraints	Description
`Id`	GUID	PK	Endpoint identifier
`UserId`	GUID	FK → Users.Id, CASCADE	Owner
`Url`	VARCHAR(2000)	Not Null	Webhook URL
`Secret`	VARCHAR(256)	—	HMAC-SHA256 signing secret
`IsActive`	BOOLEAN	Not Null	Active flag
`Description`	VARCHAR(200)	—	Description
`LastTriggeredAtUtc`	TIMESTAMP?	—	Last delivery time
`LastStatus`	VARCHAR(50)	—	Last HTTP status
`CreatedAtUtc`	TIMESTAMP	Not Null	Creation time

3.9 DeviceApiKeys

Column	Type	Constraints	Description
`Id`	GUID	PK	Key identifier
`DeviceId`	GUID	FK → Devices.Id, CASCADE	Associated device
`KeyHash`	VARCHAR(64)	Not Null	SHA256 hash of API key
`Name`	VARCHAR(100)	—	Descriptive name
`CreatedAtUtc`	TIMESTAMP	Not Null	Creation time
`LastUsedAtUtc`	TIMESTAMP?	—	Last usage time

3.10 DeviceEventLogs

Column	Type	Constraints	Description
`Id`	GUID	PK	Log entry identifier
`DeviceId`	GUID	FK → Devices.Id, CASCADE	Source device
`EventType`	INT (enum)	Not Null	Event category
`EventTimestampUtc`	TIMESTAMP	Not Null	When event occurred on device
`ReceivedAtUtc`	TIMESTAMP	Not Null	When API received event
`DataJson`	VARCHAR(8000)	—	Raw JSON payload
`Processed`	BOOLEAN	Not Null	Processing status
`ProcessingError`	VARCHAR(1000)	—	Error message if failed

Indexes:
- Composite index: `(DeviceId, EventTimestampUtc)`
- Index: `ReceivedAtUtc`

---

4. Delete Behavior Reference

Parent	Child	On Delete
User	Device	CASCADE
User	Notification	CASCADE
User	WebhookEndpoint	CASCADE
User (Caregiver)	User (Patient)	RESTRICT
User	TravelSession	CASCADE
Device	Container	CASCADE
Device	DispenseEvent	CASCADE
Device	DeviceApiKey	CASCADE
Device	DeviceEventLog	CASCADE
Device	TravelSession (Main/Portable)	RESTRICT
Container	Schedule	CASCADE
Container	DispenseEvent	RESTRICT
Container	Container (SourceCopy)	RESTRICT
Schedule	DispenseEvent	RESTRICT

---

5. Migrations

5.1 Migration History

Migration	Date	Description
`20260203104220_Initial`	Feb 3, 2026	Initial schema: Users, Devices, Containers, Schedules, DispenseEvents, TravelSessions, Notifications
`20260203143859_AddWebhooksAndApiKeys`	Feb 3, 2026	Added WebhookEndpoints, DeviceApiKeys, DeviceEventLogs tables

5.2 Migration Commands

```
# Apply all pending migrations (PostgreSQL)
cd backend
dotnet ef database update --project src/Infrastructure --startup-project src/Api

# Create a new migration
dotnet ef migrations add MigrationName --project src/Infrastructure --startup-project src/Api

# Revert last migration
dotnet ef database update PreviousMigrationName --project src/Infrastructure --startup-project src

# Generate SQL script (for manual review)
dotnet ef migrations script --project src/Infrastructure --startup-project src/Api
```

5.3 Auto-Migration on Startup

The application automatically applies migrations on startup:

```
// Program.cs — runs on every app start
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    if (useSqlite)
        await db.Database.EnsureCreatedAsync();  // SQLite: create if not exists
    else
        await db.Database.MigrateAsync();        // PostgreSQL: apply migrations
    await SeedData.SeedAsync(db);                // Seed demo data if empty
}
```

---

6. Seed Data

When the database is empty (no users), the following demo data is automatically created:

6.1 Demo Users

Email	Password	Role	Notes
patient@demo.com	Demo123!	Patient	Has caregiver assigned
caregiver@demo.com	Demo123!	Caregiver	Monitors patient
admin@demo.com	Demo123!	Admin	Full access

6.2 Demo Devices

Name	Type	Status	Owner
Home Dispenser	Main (SMD-100)	Active	Demo Patient

Name	Type	Status	Owner
Travel Dispenser	Portable (SMD-200)	Paused	Demo Patient

6.3 Demo Containers

Slot	Medication	Quantity	Pills/Dose	Low Stock	Device
1	Vitamin D	30	1	7	Home Dispenser
2	Aspirin	60	1	14	Home Dispenser

6.4 Demo Schedules

Medication	Time	Days	Notes
Vitamin D	08:00	Every day (127)	With breakfast
Aspirin	20:00	Every day (127)	After dinner

---

7. DbContext Configuration

7.1 DbSet Declarations

```
public class AppDbContext : DbContext
{
    public DbSet<User> Users => Set<User>();
    public DbSet<Device> Devices => Set<Device>();
    public DbSet<Container> Containers => Set<Container>();
    public DbSet<Schedule> Schedules => Set<Schedule>();
    public DbSet<DispenseEvent> DispenseEvents => Set<DispenseEvent>();
    public DbSet<TravelSession> TravelSessions => Set<TravelSession>();
    public DbSet<Notification> Notifications => Set<Notification>();
    public DbSet<WebhookEndpoint> WebhookEndpoints => Set<WebhookEndpoint>();
    public DbSet<DeviceApiKey> DeviceApiKeys => Set<DeviceApiKey>();
    public DbSet<DeviceEventLog> DeviceEventLogs => Set<DeviceEventLog>();
}
```

7.2 Key Configuration Highlights

- **User.Email** — Unique index, max 256 chars
- **Container.SourceContainer** — Self-referential FK for travel copies (RESTRICT delete)
- **DispenseEvent** — References Device (CASCADE), Container (RESTRICT), Schedule (RESTRICT)
- **TravelSession** — References User (CASCADE), MainDevice (RESTRICT), PortableDevice (RESTRICT)

- **DeviceEventLog** — Composite index on `(DeviceId, EventTimestampUtc)` for efficient querying

---

8. Repository Pattern

All database access goes through repository interfaces (defined in Application layer) with implementations in Infrastructure:

8.1 Repository Methods (Common Pattern)

```
// Example: IDeviceRepository
public interface IDeviceRepository
{
    Task<Device?> GetByIdAsync(Guid id, CancellationToken ct);
    Task<IReadOnlyList<Device>> GetByUserIdAsync(Guid userId, CancellationToken ct);
    Task AddAsync(Device device, CancellationToken ct);
    // ... additional query methods
}
```

8.2 Unit of Work

```
// All changes are committed atomically via UnitOfWork
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync(CancellationToken ct);
}
```

The `UnitOfWork` wraps `AppDbContext.SaveChangesAsync()`, ensuring all repository changes within a handler are committed in a single transaction.

---

9. Connection Strings

9.1 PostgreSQL (Production/Docker)

```
Host=localhost;Port=5432;Database=dispenser;Username=dispenser;Password=dispenser_secret
```

Docker Compose format:

```
Host=db;Port=5432;Database=dispenser;Username=dispenser;Password=dispenser_secret
```

9.2 SQLite (Local Development)

```
Data Source=dispenser.db
```

9.3 Switching Between Databases

Simply change the connection string in `appsettings.json` or via environment variable. The Infrastructure layer auto-detects the provider:

```
var useSqlite = conn.TrimStart().StartsWith("Data Source=", StringComparison.OrdinalIgnoreCase);
if (useSqlite)
    services.AddDbContext<AppDbContext>(o => o.UseSqlite(conn));
else
    services.AddDbContext<AppDbContext>(o => o.UseNpgsql(conn));
```

---

10. Performance Considerations

Concern	Mitigation
Large event logs	Composite index on `(DeviceId, EventTimestampUtc)`
Notification queries	Index on `ReceivedAtUtc` for recent event filtering
Unique email lookup	Unique index on `Users.Email`
Low stock check	Background job queries containers where `Quantity < LowStockThreshold`
Missed dose check	Background job queries dispense events where `Status = Pending/Dispensed` and `ScheduledAtUtc < threshold`
Connection pooling	EF Core connection pooling via PostgreSQL driver (Npgsql)

Cloud & Deployment Documentation

Smart Medication Dispenser — Docker, PostgreSQL, Environment Configuration & CI/CD

---

1. Deployment Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        DEPLOYMENT ARCHITECTURE                           │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │                    Docker Compose Cluster                        │    │
│  │                                                                  │    │
│  │   ┌─────────────────┐          ┌─────────────────┐               │    │
│  │   │   API Container │          │  DB Container   │               │    │
│  │   │                 │          │                 │               │    │
│  │   │ ASP.NET Core 8  │─────────▶│  PostgreSQL 15  │               │    │
│  │   │ Port: 5000      │   TCP    │ Port: 5432      │               │    │
│  │   │                 │   5432   │                 │               │    │
│  │   │ Auto-migrate    │          │ Volume: pgdata  │               │    │
│  │   │ Auto-seed       │          │ Health check    │               │    │
│  │   └────────┬────────┘          └─────────────────┘               │    │
│  │            │                                                      │    │
│  └────────────┼──────────────────────────────────────────────────────┘    │
│               │ Port 5000                                                 │
│               ▼                                                           │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │                    External Clients                              │    │
│  │                                                                  │    │
│  │   ┌─────────┐    ┌─────────┐    ┌─────────┐    ┌─────────┐      │    │
│  │   │ Web App │    │Mobile App│    │ ESP32   │    │ External │      │    │
│  │   │ :5173   │    │ Expo    │    │ Firmware │    │ Webhooks │      │    │
│  │   └─────────┘    └─────────┘    └─────────┘    └─────────┘      │    │
│  │                                                                  │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

---

2. Docker Configuration

2.1 Docker Compose

The project uses Docker Compose with two services:

```yaml
version: '3.8'

services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: dispenser
      POSTGRES_PASSWORD: dispenser_secret
      POSTGRES_DB: dispenser
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dispenser -d dispenser"]
      interval: 5s
      timeout: 5s
      retries: 5

  api:
    build:
      context: ./backend
      dockerfile: src/Api/Dockerfile
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      ASPNETCORE_URLS: http://+:5000
      ConnectionStrings__DefaultConnection: "Host=db;Port=5432;Database=dispenser;Username=dispens
      Jwt__SecretKey: "SmartMedicationDispenser_MVP_SecretKey_AtLeast32Characters!"
      Jwt__Issuer: "SmartMedicationDispenser"
      Jwt__Audience: "SmartMedicationDispenser"
    ports:
      - "5000:5000"
    depends_on:
      db:
        condition: service_healthy

volumes:
  pgdata:
```

2.2 Service Details

Service	Image	Port	Volume	Health Check
`db`	postgres:15-alpine	5432:5432	pgdata (persistent)	`pg_isready` every 5s
`api`	Custom (Dockerfile)	5000:5000	None	Depends on db health

2.3 Docker Commands

```
# Start all services
docker-compose up -d

# Start only database
docker-compose up -d db

# View logs
docker-compose logs -f api
docker-compose logs -f db

# Stop all services
docker-compose down

# Stop and remove volumes (DESTROYS DATA)
docker-compose down -v

# Rebuild API container after code changes
docker-compose up -d --build api

# Check service status
docker-compose ps
```

2.4 Dockerfile (Backend API)

The API Dockerfile is located at `backend/src/Api/Dockerfile` and uses a **4-stage multi-stage build**:

```
# Stage 1: Runtime base
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 5000

# Stage 2: Build
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
# Copy csproj files and restore NuGet packages
COPY ["src/Domain/Domain.csproj", "Domain/"]
COPY ["src/Application/Application.csproj", "Application/"]
COPY ["src/Infrastructure/Infrastructure.csproj", "Infrastructure/"]
COPY ["src/Api/Api.csproj", "Api/"]
RUN dotnet restore "Api/Api.csproj"
# Copy source and build
COPY src/Domain Domain/
COPY src/Application Application/
COPY src/Infrastructure Infrastructure/
COPY src/Api Api/
RUN dotnet build "Api/Api.csproj" -c Release -o /app/build

# Stage 3: Publish
FROM build AS publish
RUN dotnet publish "Api/Api.csproj" -c Release -o /app/publish /p:UseAppHost=false

# Stage 4: Final runtime image
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "Api.dll"]
```

Key optimizations:

- NuGet restore is cached (csproj files copied first)
- Build and publish stages separated for layer caching
- Final image uses slim `aspnet:8.0` runtime (not full SDK)
- `UseAppHost=false` for smaller output

---

3. PostgreSQL Database

3.1 Connection Details

Setting	Development	Docker	Production
Host	localhost	db (Docker network)	\<your-host>
Port	5432	5432	5432
Database	dispenser	dispenser	dispenser
Username	dispenser	dispenser	\<secure-user>
Password	dispenser_secret	dispenser_secret	\<secure-password>

3.2 Connection String Format

```
# PostgreSQL
Host=<host>;Port=5432;Database=dispenser;Username=dispenser;Password=dispenser_secret

# SQLite (local development)
Data Source=dispenser.db
```

3.3 Database Management

```
# Connect to PostgreSQL in Docker
docker-compose exec db psql -U dispenser -d dispenser

# Common psql commands
\dt              # List tables
\d+ Users        # Describe Users table
\l               # List databases
\q               # Quit

# Backup database
docker-compose exec db pg_dump -U dispenser dispenser > backup.sql

# Restore database
docker-compose exec -T db psql -U dispenser dispenser < backup.sql
```

3.4 PostgreSQL Configuration

The default PostgreSQL Alpine image includes sensible defaults. For production tuning:

Parameter	Default	Recommended (Prod)	Description
`max_connections`	100	200	Max concurrent connections
`shared_buffers`	128MB	256MB-1GB	Shared memory for caching
`effective_cache_size`	4GB	75% of RAM	Planner cache estimate
`work_mem`	4MB	16MB	Memory per sort/hash operation
`maintenance_work_mem`	64MB	256MB	Memory for VACUUM, CREATE INDEX

---

4. Environment Configuration

4.1 Backend Environment Variables

Variable	Required	Default	Description
`ASPNETCORE_ENVIRONMENT`	No	Development	Runtime environment (Development, Production)
`ASPNETCORE_URLS`	No	http://localhost:5000	Listen URLs
`ConnectionStrings__DefaultConnection`	Yes	PostgreSQL localhost	Database connection string
`Jwt__SecretKey`	Yes	Dev default key	JWT signing secret (min 32 chars)
`Jwt__Issuer`	No	SmartMedicationDispenser	JWT issuer claim
`Jwt__Audience`	No	SmartMedicationDispenser	JWT audience claim

4.2 Web Portal Environment (.env)

```
# web/.env
VITE_API_URL=http://localhost:5000
```

Variable	Default	Description
`VITE_API_URL`	(empty — uses proxy)	Backend API base URL

Note: In development, Vite proxies `/api` requests to `http://localhost:5000` (configured in `vite.config.ts`), so `VITE_API_URL` can be empty.

4.3 Mobile App Environment (.env)

```
# mobile/.env
EXPO_PUBLIC_API_URL=http://localhost:5000
```

Variable	Default	Description
`EXPO_PUBLIC_API_URL`	http://localhost:5000	Backend API base URL

Note for physical devices: Use your machine's local IP (e.g., `http://192.168.1.100:5000`) instead of `localhost`.

---

5. Development Setup

5.1 Option A: Full Docker Stack

```
cd smart-medication-dispenser
docker-compose up -d

# API ready at http://localhost:5000
# Swagger at http://localhost:5000/swagger
# PostgreSQL at localhost:5432
```

5.2 Option B: Database in Docker, API Local

```
# Start only PostgreSQL
docker-compose up -d db

# Run API locally
cd backend
dotnet restore
dotnet run --project src/Api

# API at http://localhost:5000
```

5.3 Option C: Fully Local (SQLite)

```
# No Docker needed — use SQLite
cd backend

# Update connection string in appsettings.Development.json:
# "DefaultConnection": "Data Source=dispenser.db"

dotnet restore
dotnet run --project src/Api
```

5.4 Running Frontend Applications

```
# Web portal (separate terminal)
cd web
cp .env.example .env
npm install
npm run dev
# → http://localhost:5173

# Mobile app (separate terminal)
cd mobile
cp .env.example .env
npm install
npx expo start
# → Scan QR code with Expo Go
```

6. Production Deployment Guide

6.1 Security Checklist

Item	Action	Priority
JWT Secret	Use cryptographically random 64+ character key	Critical
DB Password	Use strong, unique password	Critical
HTTPS	Enable TLS termination (reverse proxy or cloud LB)	Critical
CORS	Restrict to specific frontend domains	High
API Rate Limiting	Add rate limiting middleware	High
DB Backup	Configure automated backups	High
Logging	Configure structured logging (Serilog → ELK/Seq)	Medium
Health Checks	Add `/health` endpoint for monitoring	Medium
Secrets Management	Use Azure Key Vault / AWS Secrets Manager / HashiCorp Vault	High

6.2 Production docker-compose.yml Adjustments

```
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: dispenser
    # Remove port mapping in production (access only from API container)
    # ports:
    #   - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: always

  api:
    build:
      context: ./backend
      dockerfile: src/Api/Dockerfile
    environment:
      ASPNETCORE_ENVIRONMENT: Production
      ASPNETCORE_URLS: http://+:5000
      ConnectionStrings__DefaultConnection: "Host=db;Port=5432;Database=dispenser;Username=${DB_US
      Jwt__SecretKey: ${JWT_SECRET}
    ports:
      - "5000:5000"
    depends_on:
      db:
        condition: service_healthy
    restart: always
```

6.3 Reverse Proxy (Nginx)

For production, place Nginx in front of the API:

```
server {
    listen 443 ssl http2;
    server_name api.smartdispenser.ch;

    ssl_certificate /etc/ssl/certs/smartdispenser.crt;
    ssl_certificate_key /etc/ssl/private/smartdispenser.key;

    location / {
        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

---

7. Cloud Provider Deployment Options

7.1 Azure (Recommended for .NET)

Service	Resource	Purpose
Azure App Service	B1-S1 tier	Host ASP.NET Core API
Azure Database for PostgreSQL	Flexible Server	Managed PostgreSQL
Azure Key Vault	—	Secrets management
Azure Container Registry	—	Docker image storage
Azure Application Insights	—	Monitoring & telemetry

7.2 AWS

Service	Resource	Purpose
ECS Fargate or Elastic Beanstalk	—	Host API container
RDS PostgreSQL	db.t3.micro+	Managed PostgreSQL
AWS Secrets Manager	—	Secrets management
ECR	—	Docker image storage
CloudWatch	—	Logging & monitoring

7.3 Google Cloud

Service	Resource	Purpose
Cloud Run	—	Serverless API container
Cloud SQL for PostgreSQL	—	Managed PostgreSQL
Secret Manager	—	Secrets management
Artifact Registry	—	Docker image storage

---

8. Monitoring & Logging

8.1 Application Logging

ASP.NET Core uses built-in `ILogger` with configurable levels:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning"
    }
  }
}
```

8.2 Log Categories

Category	Level	What's Logged
API requests	Information	Request/response timing
Device heartbeats	Debug	Each heartbeat received
Device events	Information	Event processing
Webhook delivery	Information	POST URL + status code
Webhook failures	Warning	POST failures
Device errors	Error	Device-reported errors
Background jobs	Information/Error	Missed dose checks, low stock
Unhandled exceptions	Error	Full exception details

8.3 Docker Logs

```
# Follow API logs
docker-compose logs -f api

# Follow DB logs
docker-compose logs -f db

# Last 100 lines
docker-compose logs --tail 100 api

# Export logs to file
docker-compose logs api > api_logs.txt
```

---

9. Backup & Recovery

9.1 Database Backup

```
# Full backup
docker-compose exec db pg_dump -U dispenser -Fc dispenser > backup_$(date +%Y%m%d).dump

# Plain SQL backup
docker-compose exec db pg_dump -U dispenser dispenser > backup_$(date +%Y%m%d).sql

# Compressed backup
docker-compose exec db pg_dump -U dispenser -Fc -Z 9 dispenser > backup_$(date +%Y%m%d).dump.gz
```

9.2 Database Restore

```
# Restore from custom format
docker-compose exec -T db pg_restore -U dispenser -d dispenser --clean < backup.dump

# Restore from SQL
docker-compose exec -T db psql -U dispenser dispenser < backup.sql
```

9.3 Automated Backup Script

```
#!/bin/bash
# backup.sh - Run daily via cron
BACKUP_DIR="/backups/dispenser"
DATE=$(date +%Y%m%d_%H%M%S)

docker-compose exec -T db pg_dump -U dispenser -Fc dispenser > "${BACKUP_DIR}/dispenser_${DATE}.du

# Keep only last 30 days
find "${BACKUP_DIR}" -name "dispenser_*.dump" -mtime +30 -delete
```

---

10. Scaling Considerations

Aspect	Current (MVP)	Production Recommendation
API instances	1 container	2-4 behind load balancer
Database	Single PostgreSQL	Managed PostgreSQL with read replicas
Background jobs	In-process hosted service	Extract to separate worker service
Webhook delivery	Synchronous in background job	Message queue (RabbitMQ/Azure Service Bus)
Caching	None	Redis for session/frequent queries
Static assets	Served by API	CDN for web portal assets
File storage	Local SQLite DB file	Azure Blob / S3 for images

Web Portal Documentation

Smart Medication Dispenser — React + TypeScript + Vite Caregiver Portal

---

1. Overview

The web portal is a **single-page application (SPA)** built for caregivers and administrators to manage medication dispensers, monitor patient adherence, and configure integrations.

Aspect	Detail
Framework	React 18
Language	TypeScript
Build Tool	Vite
Styling	Tailwind CSS 3.x
UI Components	shadcn/ui (Radix UI primitives)
Routing	React Router v6 (BrowserRouter)
HTTP Client	Axios
State Management	React Context (AuthContext)
Charts	Recharts
Animations	Motion (Framer Motion)
Icons	Lucide React
Form Validation	React Hook Form + Zod
Toast Notifications	Sonner
Dev Server	http://localhost:5173

---

2. Project Structure

```
web/
├── index.html                    # HTML entry point
├── package.json                  # Dependencies & scripts
├── tsconfig.json                 # TypeScript configuration
├── vite.config.ts                # Vite build & dev server config
├── tailwind.config.js            # Tailwind CSS configuration
├── postcss.config.js             # PostCSS plugins
├── .env                          # Environment variables
├── .env.example                  # Example env file
│
├── public/
│   └── vite.svg                  # Static assets
│
└── src/
    ├── main.tsx                  # React DOM render entry
    ├── App.tsx                   # Root component (routing)
    ├── index.css                 # Global styles + Tailwind imports
    ├── vite-env.d.ts             # Vite type definitions
    │
    ├── api/
    │   └── client.ts             # Axios instance + all API functions
    │
    ├── pages/                    # 11 page components
    │   ├── Dashboard.tsx         # Home dashboard with adherence & today's schedule
    │   ├── Devices.tsx           # Device list management
    │   ├── DeviceDetail.tsx      # Single device detail view
    │   ├── Containers.tsx        # Container (slot) management per device
    │   ├── Schedules.tsx         # Schedule management per container
    │   ├── History.tsx           # Dispense event history
    │   ├── Travel.tsx            # Travel mode management
    │   ├── Notifications.tsx     # Notification center
    │   ├── Integrations.tsx      # Webhooks & API key management
    │   ├── Login.tsx             # Login page
    │   └── Register.tsx          # Registration page
    │
    ├── components/
    │   ├── Layout.tsx            # Main layout with sidebar navigation
    │   └── ProtectedRoute.tsx    # Auth guard (redirects to /login)
    │
    ├── app/components/
    │   ├── figma/
    │   │   └── ImageWithFallback.tsx
    │   └── ui/                   # shadcn/ui components
    │       ├── alert-dialog.tsx
    │       ├── badge.tsx
    │       ├── button.tsx
    │       ├── card.tsx
    │       ├── input.tsx
    │       ├── label.tsx
    │       ├── progress.tsx
    │       ├── select.tsx
    │       ├── sonner.tsx
    │       ├── tabs.tsx
    │       ├── textarea.tsx
    │       └── utils.ts
    │
    ├── contexts/
    │   └── AuthContext.tsx        # Authentication provider
```

```
    │
    ├── lib/
    │   ├── deviceTypes.ts        # Device type constants
    │   └── mockData.ts           # Mock data for development
    │
    ├── styles/
    │   ├── theme.css             # CSS custom properties (colors, sizes)
    │   └── responsive.css        # Mobile-responsive breakpoints
    │
    └── utils/
        └── apiError.ts           # API error message extraction
```

---

3. Routing

3.1 Route Map

```
/                        → Dashboard (protected)
├── /login               → Login page (public)
├── /register            → Registration page (public)
├── /devices             → Device list (protected)
│   └── /devices/:deviceId           → Device detail (protected)
│       └── /devices/:deviceId/containers → Containers for device (protected)
├── /containers/:containerId/schedules → Schedules for container (protected)
├── /history             → Dispense event history (protected)
├── /travel              → Travel mode management (protected)
├── /notifications       → Notification center (protected)
├── /integrations        → Webhooks & API keys (protected)
└── /*                   → Redirect to / (catch-all)
```

3.2 Route Configuration

```
// App.tsx — simplified route structure
<Routes>
  {/* Public routes */}
  <Route path="/login" element={<Login />} />
  <Route path="/register" element={<Register />} />

  {/* Protected routes (require JWT) */}
  <Route path="/" element={<ProtectedRoute><Layout /></ProtectedRoute>}>
    <Route index element={<Dashboard />} />
    <Route path="devices" element={<Devices />} />
    <Route path="devices/:deviceId" element={<DeviceDetail />} />
    <Route path="devices/:deviceId/containers" element={<Containers />} />
    <Route path="containers/:containerId/schedules" element={<Schedules />} />
    <Route path="history" element={<History />} />
    <Route path="travel" element={<Travel />} />
    <Route path="notifications" element={<Notifications />} />
    <Route path="integrations" element={<Integrations />} />
  </Route>

  {/* Catch-all */}
  <Route path="*" element={<Navigate to="/" replace />} />
</Routes>
```

3.3 Protected Route Component

```
// ProtectedRoute.tsx
// If no JWT token in context → redirect to /login
// Otherwise → render children
```

---

4. Authentication (AuthContext)

4.1 Authentication Flow

The web portal's AuthContext currently uses **mock data** for login/register (via `mockUsers` in `lib/mockData.ts`). The real API client (`api/client.ts`) is fully implemented with Axios interceptors and can be switched to live API by updating the AuthContext.

Current (mock) flow:

```
1. User enters email/password on Login page
2. AuthContext checks mockUsers for matching credentials
3. Generates a mock JWT token string
4. Stores token as 'jwt_token' and user data as 'user_data' in localStorage
5. On page load → checks localStorage for existing token/user data
```

Production flow (using api/client.ts):

```
1. User enters email/password on Login page
2. POST /api/auth/login → returns { token, email, fullName, role, userId }
3. AuthContext stores token in localStorage
4. Axios interceptor adds "Authorization: Bearer <token>" to all requests
5. On 401 response → clear token → redirect to /login
6. On page load → check localStorage for existing token → GET /api/auth/me to validate
```

> **Note:** The `api/client.ts` file contains fully typed API functions for all endpoints. The mock AuthContext can be replaced with real API calls by updating `contexts/AuthContext.tsx` to use `authApi.login()` / `authApi.register()` / `authApi.me()` instead of mock data.

4.2 AuthContext API

Property/Method	Type	Description
`user`	User | null	Current user info (id, email, fullName, role)
`loading`	boolean	True while checking auth on mount

Property/Method	Type	Description
`login(email, password)`	async function	Login and store token + user data
`register(email, password, fullName, role)`	async function	Register and store token + user data
`logout()`	function	Clear token, clear user data

4.3 Token Management

- **Storage Keys:** `jwt_token` (token string), `user_data` (JSON user object) in `localStorage`
- **Axios Interceptor (api/client.ts):** Automatically attaches `Bearer <token>` from `localStorage.token` key
- **401 Handling (api/client.ts):** Clears token and redirects to `/login`

> **Important:** The mock AuthContext uses `localStorage.jwt_token` while the Axios client reads from `localStorage.token`. When switching to real API calls, ensure consistent key names.

---

5. API Client

5.1 Axios Configuration

```
// api/client.ts
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '',
  headers: { 'Content-Type': 'application/json' },
});

// Request interceptor: attach JWT
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Response interceptor: handle 401
api.interceptors.response.use(
  (r) => r,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);
```

5.2 API Function Groups

Group	Functions	Description
`authApi`	register, login, me, meProfile	Authentication
`devicesApi`	list, get, create, pause, resume, heartbeat	Device management
`containersApi`	listByDevice, create, update, delete	Container CRUD
`schedulesApi`	listByContainer, create, update, delete, today	Schedule management
`dispensingApi`	dispense, confirm, delay	Dose lifecycle
`travelApi`	start, end	Travel mode
`historyApi`	events	Event history
`adherenceApi`	me	Adherence statistics
`notificationsApi`	list, markRead	Notifications
`integrationsApi`	getWebhooks, createWebhook, deleteWebhook, getDeviceApiKeys, createDeviceApiKey, deleteDeviceApiKey	Integrations

5.3 TypeScript Types (API Contract)

All request/response types are defined in `api/client.ts`:

Type	Properties
`AuthResponse`	token, email, fullName, role, userId
`DeviceDto`	id, userId, name, type, status, timeZoneId, lastHeartbeatAtUtc
`ContainerDto`	id, deviceId, slotNumber, medicationName, quantity, pillsPerDose, lowStockThreshold
`ScheduleDto`	id, containerId, timeOfDay, daysOfWeekBitmask, startDate, endDate, notes
`DispenseEventDto`	id, deviceId, containerId, scheduleId, scheduledAtUtc, status, timestamps
`NotificationDto`	id, type, title, body, isRead, createdAtUtc
`AdherenceSummaryDto`	totalScheduled, confirmed, missed, pending, adherencePercent
`TravelSessionDto`	id, mainDeviceId, portableDeviceId, startedAtUtc, endedAtUtc, plannedEndDateUtc
`WebhookEndpointDto`	id, url, isActive, description, lastTriggeredAtUtc, lastStatus
`DeviceApiKeyDto`	id, name, createdAtUtc, lastUsedAtUtc

---

6. Pages

6.1 Dashboard

Route: `/` (index)

Features:
- Adherence summary (percentage, confirmed/missed/pending counts)
- Today's medication schedule for selected device
- Quick actions (dispense, confirm, view history)
- Device status overview

6.2 Devices

Route: `/devices`

Features:
- List all user's devices with status badges
- Create new device (Main or Portable)
- Pause/Resume device dispensing
- Navigate to device detail / containers

6.3 Device Detail

Route: `/devices/:deviceId`

Features:
- Device information (name, type, status, last heartbeat)
- Hardware status (battery, WiFi signal, temperature, humidity)
- Container overview for this device
- Recent dispense events

6.4 Containers

Route: `/devices/:deviceId/containers`

Features:
- List all medication slots for a device
- Create container (slot number, medication name, quantity, pills per dose)
- Update container (refill quantity, change medication)
- Delete container (only if no schedules reference it)
- Low stock indicator

6.5 Schedules

Route: `/containers/:containerId/schedules`

Features:
- List schedules for a container
- Create schedule (time of day, days of week, date range, notes)
- Edit schedule
- Delete schedule
- Days of week bitmask picker

6.6 History

Route: `/history`

Features:
- Dispense event timeline with status badges
- Filter by date range (from/to)
- Filter by device
- Status colors: Confirmed (green), Missed (red), Pending (yellow), Delayed (orange)

6.7 Travel

Route: `/travel`

Features:
- Start travel session (select portable device, set duration)
- View active travel session
- End travel session
- Container copy status

6.8 Notifications

Route: `/notifications`

Features:
- Notification list with type badges
- Mark as read
- Notification types: Missed Dose, Low Stock, Travel, Device errors
- Unread count indicator

6.9 Integrations

Route: `/integrations`

Features:
- **Outgoing Webhooks:** Create, list, delete webhook endpoints
- **Device API Keys:** Create per device, list, revoke
- API key shown only once on creation (copy prompt)
- Webhook delivery status (last triggered, HTTP status)

6.10 Login

Route: `/login`

Features:
- Email/password form
- Error messages for invalid credentials
- Link to registration
- Auto-redirect if already logged in

6.11 Register

Route: `/register`

Features:
- Email, password, full name, role selection
- Role picker (Patient, Caregiver)
- Validation errors
- Auto-login after successful registration

---

7. UI Components & Libraries

7.1 shadcn/ui Components (Radix UI Primitives)

Component	File	Usage
AlertDialog	`alert-dialog.tsx`	Confirmation dialogs (delete actions)
Badge	`badge.tsx`	Status badges (Active, Paused, Missed, etc.)
Button	`button.tsx`	All clickable actions
Card	`card.tsx`	Content containers
Input	`input.tsx`	Text inputs
Label	`label.tsx`	Form labels
Progress	`progress.tsx`	Adherence percentage bar
Select	`select.tsx`	Dropdowns (device type, role, etc.)
Sonner (Toaster)	`sonner.tsx`	Toast notifications
Tabs	`tabs.tsx`	Tab navigation within pages
Textarea	`textarea.tsx`	Multi-line text input

7.2 Additional Libraries

Library	Package	Usage
Recharts	`recharts`	Dashboard charts (AreaChart, RadialBarChart, PieChart)
Motion	`motion` (Framer Motion)	Page transitions, hover animations, layout animations
Lucide React	`lucide-react`	Icon library (Pill, Box, Bell, Clock, etc.)
React Hook Form	`react-hook-form` + `@hookform/resolvers`	Form state management
Zod	`zod`	Schema validation for forms
Class Variance Authority	`class-variance-authority`	Component variant styling
Tailwind Merge	`tailwind-merge` + `clsx`	Class name merging utilities

---

8. Styling

8.1 Tailwind CSS

Configuration in `tailwind.config.js` extends default theme with project-specific colors and spacing.

8.2 Theme CSS

Custom CSS properties in `styles/theme.css` define the color palette:

- Primary colors (green tones matching brand)
- Semantic colors (success, warning, error)
- Surface colors (backgrounds, borders)
- Typography scale

8.3 Responsive Design

`styles/responsive.css` handles mobile breakpoints:

Breakpoint	Width	Target
`sm`	≥640px	Mobile landscape
`md`	≥768px	Tablet
`lg`	≥1024px	Desktop
`xl`	≥1280px	Large desktop

---

9. Build & Development

9.1 Vite Configuration

```
// vite.config.ts
export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: { '@': path.resolve(__dirname, './src') },
  },
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:5000',
        changeOrigin: true,
      },
    },
  },
});
```

Key features:
- **Path alias:** `@/` maps to `src/` for clean imports
- **API proxy:** `/api/*` requests proxied to backend at `localhost:5000`
- **Hot Module Replacement (HMR):** Instant updates during development

9.2 Commands

```
# Install dependencies
npm install

# Development server (http://localhost:5173)
npm run dev

# Production build
npm run build

# Preview production build
npm run preview

# Type checking
npx tsc --noEmit
```

9.3 Build Output

```
npm run build
# → dist/
#   ├── index.html
#   └── assets/
#       ├── index-[hash].js
#       └── index-[hash].css
```

---

10. Development Workflow

10.1 Adding a New Page

1. Create component in `src/pages/NewPage.tsx`
2. Add route in `src/App.tsx` under the protected `<Route>` group
3. Add navigation link in `src/components/Layout.tsx`
4. If new API calls needed, add functions in `src/api/client.ts`

10.2 Adding a New API Call

1. Define TypeScript types in `src/api/client.ts`
2. Add API function to the appropriate group (e.g., `devicesApi`)
3. Use in page component: `const { data } = await devicesApi.list()`

10.3 Adding a New UI Component

1. Copy shadcn/ui component to `src/app/components/ui/`
2. Import in page: `import { Button } from '@/app/components/ui/button'`

Mobile App Documentation

Smart Medication Dispenser — React Native / Expo Patient Application

---

1. Overview

The mobile app is built for **patients** to manage their daily medication, receive reminders, confirm dose intake, and view their dispense history.

Aspect	Detail
Framework	React Native 0.76.5
Platform	Expo ~52.0.0 (managed workflow)
Language	TypeScript
Navigation	Expo Router ~4.0.0 (file-based routing)
HTTP Client	Axios
State Management	React Context (AuthContext)
Token Storage	AsyncStorage (persistent across app restarts)
Notifications	Expo Notifications (local)
Target Platforms	iOS, Android
Bundle ID (iOS)	com.smartdispenser.mobile
Package (Android)	com.smartdispenser.mobile

---

2. Project Structure

```
mobile/
├── app.json                      # Expo configuration
├── package.json                  # Dependencies & scripts
├── tsconfig.json                 # TypeScript configuration
├── babel.config.js               # Babel configuration
├── .env.example                  # Example environment variables
│
├── app/                          # Expo Router pages (file-based routing)
│   ├── _layout.tsx               # Root layout (AuthProvider wrapper)
│   ├── index.tsx                 # Entry redirect (→ login or home)
│   ├── login.tsx                 # Login screen
│   ├── register.tsx              # Registration screen
│   │
│   ├── (tabs)/                   # Tab navigation group
│   │   ├── _layout.tsx           # Tab bar configuration
│   │   ├── home.tsx              # Today's schedule & quick actions
│   │   ├── devices.tsx           # Device list & management
│   │   ├── history.tsx           # Dispense event history
│   │   └── notifications.tsx     # Notification center
│   │
│   └── dose/
│       └── [id].tsx              # Dose detail / confirmation screen
│
└── src/
    ├── api/
    │   ├── client.ts             # Axios instance + all API functions
    │   └── deviceEvents.ts       # Device event type utilities
    │
    ├── context/
    │   └── AuthContext.tsx        # Authentication provider
    │
    └── notifications/
        └── scheduleNotifications.ts  # Local notification scheduling
```

3. Navigation Architecture

3.1 Expo Router (File-Based Routing)

```
app/
├── _layout.tsx       → Root layout (wraps everything in AuthProvider)
├── index.tsx         → "/" — Redirect based on auth state
├── login.tsx         → "/login" — Public login screen
├── register.tsx      → "/register" — Public registration screen
│
├── (tabs)/           → Tab group (protected)
│   ├── _layout.tsx   → Tab bar with 4 tabs
│   ├── home.tsx      → "/(tabs)/home" — Today's schedule
│   ├── devices.tsx   → "/(tabs)/devices" — Device list
│   ├── history.tsx   → "/(tabs)/history" — Event history
│   └── notifications.tsx → "/(tabs)/notifications" — Alerts
│
└── dose/
    └── [id].tsx      → "/dose/:id" — Dose detail (dynamic route)
```

3.2 Navigation Flow

```
App Launch
    │
    ├── Has token? → Validate with GET /api/auth/me
    │       │
    │       ├── Valid → Navigate to /(tabs)/home
    │       └── Invalid → Clear token → /login
    │
    └── No token → /login
          │
          ├── Login success → /(tabs)/home
          └── Register link → /register
                └── Register success → /(tabs)/home
```

3.3 Tab Bar Configuration

Tab	Icon	Screen	Description
Home	🏠	`home.tsx`	Today's schedule, quick dispense/confirm
Devices	📱	`devices.tsx`	Device list, status, pause/resume
History	📋	`history.tsx`	Past dispense events timeline
Notifications	🔔	`notifications.tsx`	Alerts with unread badge

---

4. Screens

4.1 Home Screen (`home.tsx`)

Primary screen for daily medication management.

Features:
- Today's medication schedule (sorted by time)
- For each scheduled dose:
- Medication name and slot number
- Scheduled time
- Status badge (Pending, Dispensed, Confirmed, Missed)
- "Dispense" button (triggers physical dispense)
- "Confirm" button (marks dose as taken)
- "Delay" button (snooze reminder)
- Pull-to-refresh for latest schedule
- Device selector (if user has multiple devices)

API Calls:
- `schedulesApi.today(deviceId, timeZoneId)` — Get today's schedule
- `dispensingApi.dispense(deviceId, { scheduleId })` — Trigger dispense
- `dispensingApi.confirm(eventId)` — Confirm intake
- `dispensingApi.delay(eventId, { minutes })` — Delay reminder

4.2 Devices Screen (`devices.tsx`)

Device management and status monitoring.

Features:
- List all user's devices with status indicators
- Device type badge (Main / Portable)
- Connection status (online/offline)
- Battery level (portable devices)
- WiFi signal strength
- Pause/Resume toggle
- Last heartbeat timestamp

API Calls:
- `devicesApi.list()` — List all devices
- `devicesApi.pause(id)` — Pause device
- `devicesApi.resume(id)` — Resume device

4.3 History Screen (`history.tsx`)

Dispense event history and adherence tracking.

Features:
- Timeline of dispense events
- Status-colored entries:
- Green: Confirmed (taken)
- Red: Missed
- Yellow: Pending
- Orange: Delayed
- Medication name and time
- Date filter (today, this week, custom range)
- Device filter

API Calls:
- `historyApi.events(deviceId, { fromUtc, toUtc, limit })` — Get event history

4.4 Notifications Screen (`notifications.tsx`)

Alert center for medication and device notifications.

Features:
- Notification list sorted by date (newest first)
- Type badges (Missed Dose, Low Stock, Device Error, etc.)
- Unread/read status
- Tap to mark as read
- Notification detail view

API Calls:
- `notificationsApi.list(limit)` — Get notifications
- `notificationsApi.markRead(id)` — Mark as read

4.5 Dose Detail Screen (`dose/[id].tsx`)

Dynamic route for individual dose confirmation.

Route: `/dose/:id` (where `id` is the dispense event ID)

Features:
- Medication details (name, pills per dose, slot)
- Scheduled time and current status
- Large "Confirm Taken" button
- "Delay" option with minute selector
- Status history for this dose

API Calls:
- `dispensingApi.confirm(eventId)` — Confirm intake
- `dispensingApi.delay(eventId, { minutes })` — Delay

4.6 Login Screen (`login.tsx`)

Authentication entry point.

Features:
- Email and password fields
- Login button
- "Create account" link to register
- Error display for invalid credentials
- Loading state during API call

4.7 Register Screen (`register.tsx`)

New user registration.

Features:
- Email, password, full name fields
- Role selection (Patient)
- Register button
- "Already have account" link to login
- Validation error display

---

5. Authentication

5.1 AuthContext

```
// context/AuthContext.tsx
interface AuthContextType {
  token: string | null;
  user: { id: string; email: string; fullName: string; role: string } | null;
  login: (email: string, password: string) => Promise<void>;
  register: (email: string, password: string, fullName: string) => Promise<void>;
  logout: () => void;
  loading: boolean;
}
```

5.2 Token Storage

The mobile app uses **two-layer token storage**:

1. **AsyncStorage (persistent):** Token persisted to device storage via `@react-native-async-storage/async-storage` under key `"token"`
2. **In-memory (runtime):** Token also stored in `authToken` variable in `api/client.ts` for Axios interceptor

On app launch:

```
1. Read token from AsyncStorage
2. If token exists → set authToken in api/client.ts
3. Call GET /api/auth/me to validate token
4. If valid → set user state, navigate to Home
5. If invalid → clear AsyncStorage, navigate to Login
```

On login/register:

```
1. Call POST /api/auth/login (or /register)
2. Store token in AsyncStorage (persistent)
3. Set authToken in api/client.ts (in-memory)
4. Set user state from response
```

On logout:

```
1. Remove token from AsyncStorage
2. Clear authToken in api/client.ts
3. Clear user state → navigates to Login
```

5.3 API Client Authentication

```
// api/client.ts
export let authToken: string | null = null;
export function setAuthToken(token: string | null) {
  authToken = token;
}

// Request interceptor adds Bearer token
api.interceptors.request.use((config) => {
  if (authToken) config.headers.Authorization = `Bearer ${authToken}`;
  return config;
});

// 401 response clears token
api.interceptors.response.use(
  (r) => r,
  (err) => {
    if (err.response?.status === 401) authToken = null;
    return Promise.reject(err);
  }
);
```

---

6. API Client

6.1 Base URL Configuration

```
const baseURL = Constants.expoConfig?.extra?.apiUrl
  ?? process.env.EXPO_PUBLIC_API_URL
  ?? 'http://localhost:5000';
```

Priority:

1. Expo config extra (`app.json` → `expo.extra.apiUrl`)
2. Environment variable (`EXPO_PUBLIC_API_URL`)
3. Fallback: `http://localhost:5000`

6.2 Available API Functions

Group	Methods
`authApi`	register, login, me, meProfile
`devicesApi`	list, get, pause, resume
`schedulesApi`	today
`dispensingApi`	dispense, confirm, delay
`historyApi`	events
`notificationsApi`	list, markRead

6.3 Error Handling

```
// Utility function to extract user-friendly error messages
export function getApiErrorMessage(err: unknown): string {
  // Checks: response.data.detail → response.data.message →
  //         response.data.errors (first) → generic message
}
```

6.4 Device DTO (Extended for Mobile)

The mobile client includes additional device fields:

```
export type DeviceDto = {
  id: string;
  userId: string;
  name: string;
  type: 'Main' | 'Portable';
  status: 'Active' | 'Paused';
  timeZoneId?: string;
  lastHeartbeatAtUtc?: string;
  firmwareVersion?: string;
  isOnline: boolean;
  batteryLevel?: number;     // 0-100
  wifiSignal?: number;       // dBm
  temperature?: number;      // Celsius
  humidity?: number;          // 0-100
};
```

---

7. Notifications

7.1 Local Notifications

The app uses **Expo Notifications** for local push notifications via `scheduleNotifications.ts`:

Notification Handler (foreground):

```
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: true,
  }),
});
```

Android Channel: `dose-reminders` (HIGH importance)

Scheduling Flow:

```
1. Home screen loads → calls scheduleDoseNotifications(todayItems, deviceId)
2. Cancels ALL previously scheduled notifications
3. Requests notification permission (creates Android channel if needed)
4. For each future dose → schedules local notification with:
   - Title: "Time for medication"
   - Body: "{medicationName} – {pillsPerDose} pill(s)"
   - Data: { scheduleId, containerId, deviceId, medicationName, pillsPerDose }
```

Notification Tap Handler (in root _layout.tsx):

```
When user taps notification:
  → addNotificationResponseListener() extracts data payload
  → Navigates to /dose/[id] with scheduleId, containerId, deviceId params
```

7.2 Notification Types

Type	Trigger	Priority
Dose Reminder	Scheduled dose time (future only)	High
Missed Dose	Server-side (background hosted service)	Critical
Low Stock	Server-side (background hosted service)	Medium

> **Note:** Only dose reminders are local notifications. Missed dose and low stock alerts come from the server as in-app notifications via the `/api/notifications` endpoint.

7.3 Expo Configuration

```
// app.json
{
  "expo": {
    "plugins": ["expo-router", "expo-notifications"]
  }
}
```

---

8. Build & Development

8.1 Setup Commands

```
# Install dependencies
cd mobile
npm install

# Copy environment file
cp .env.example .env

# Edit .env with your API URL
# EXPO_PUBLIC_API_URL=http://192.168.1.100:5000
```

8.2 Development

```
# Start Expo dev server
npx expo start

# Options:
# Press 'i' for iOS simulator
# Press 'a' for Android emulator
# Scan QR code with Expo Go app on physical device
```

8.3 Important: API URL for Physical Devices

When running on a physical device, `localhost` won't work. Use your machine's IP:

```
# Find your IP
# Windows: ipconfig
# Mac/Linux: ifconfig

# Set in .env:
EXPO_PUBLIC_API_URL=http://192.168.1.100:5000
```

8.4 Building for Production

```
# Build for iOS
npx expo build:ios

# Build for Android
npx expo build:android

# Or with EAS Build
npx eas build --platform ios
npx eas build --platform android
```

---

9. Expo Configuration

9.1 app.json

```
{
  "expo": {
    "name": "Smart Medication Dispenser",
    "slug": "smart-medication-dispenser",
    "version": "1.0.0",
    "orientation": "portrait",
    "scheme": "smartdispenser",
    "userInterfaceStyle": "automatic",
    "ios": {
      "supportsTablet": true,
      "bundleIdentifier": "com.smartdispenser.mobile"
    },
    "android": {
      "package": "com.smartdispenser.mobile"
    },
    "plugins": ["expo-router", "expo-notifications"]
  }
}
```

9.2 Key Configuration

Setting	Value	Description
`scheme`	smartdispenser	Deep link URL scheme
`userInterfaceStyle`	automatic	Follows system dark/light mode
`orientation`	portrait	Lock to portrait mode
`plugins`	expo-router, expo-notifications	Required Expo plugins

---

10. Dependencies

10.1 Key Dependencies

Package	Version	Purpose
`expo`	~52.0.0	Expo framework
`react-native`	0.76.5	Mobile UI framework
`expo-router`	~4.0.0	File-based navigation
`expo-notifications`	~0.29.9	Local push notifications
`@react-native-async-storage/async-storage`	1.23.1	Persistent token storage
`axios`	^1.7.7	HTTP client
`expo-constants`	~17.0.3	Access app configuration
`expo-linking`	~7.0.3	Deep linking
`expo-splash-screen`	~0.29.18	Splash screen management
`react-native-safe-area-context`	4.12.0	Safe area insets
`react-native-screens`	~4.4.0	Native screen containers

10.2 Package Scripts

```
{
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web"
  }
}
```

---

11. Known Limitations (MVP)

Limitation	Description	Future Plan
Token in AsyncStorage	Uses AsyncStorage (not encrypted)	Migrate to Expo SecureStore for encrypted storage
Local notifications only	No FCM/APNs push server	Integrate Firebase Cloud Messaging
No offline support	Requires network for all actions	Add local cache + sync queue
No biometric auth	Password-only login	Add fingerprint/face authentication
No image upload	Medication images are URL-only	Add camera/gallery image upload
Single language	English only	Add i18n localization

Authentication & Authorization

Smart Medication Dispenser — JWT, API Keys, User Roles & Security

---

1. Overview

The platform uses a **dual authentication system**:

Auth Method	Use Case	Token Type	Consumers
JWT Bearer	User authentication	HS256 JWT (7-day expiry)	Web portal, Mobile app
X-API-Key	Machine-to-machine	SHA256-hashed API key	ESP32 firmware, External systems

```
┌──────────────────────────────────────────────────────────────────┐
│                  AUTHENTICATION ARCHITECTURE                     │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  ┌──────────────┐   JWT Bearer          ┌──────────────────┐    │
│  │ Web Portal   │──────────────────────▶│                  │    │
│  │ Mobile App   │  Authorization:       │   ASP.NET Core   │    │
│  └──────────────┘  Bearer <token>       │                  │    │
│                                         │  ┌────────────┐   │    │
│  ┌──────────────┐   X-API-Key           │  │ JWT Auth   │   │    │
│  │ ESP32 Device │──────────────────────▶│  │ Middleware │   │    │
│  │ Cloud System │  X-API-Key: sk_...    │  └────────────┘   │    │
│  └──────────────┘                       │  ┌────────────┐   │    │
│                                         │  │ API Key    │   │    │
│                                         │  │ Resolver   │   │    │
│                                         │  └────────────┘   │    │
│                                         └──────────────────┘    │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

---

2. JWT Authentication

2.1 Token Generation

JWT tokens are generated by `JwtAuthService` on login or registration:

```
// JwtAuthService.cs
public string GenerateJwt(Guid userId, string email, UserRole role)
{
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.SecretKey));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, userId.ToString()),
        new Claim(ClaimTypes.Email, email),
        new Claim(ClaimTypes.Role, role.ToString())
    };

    var token = new JwtSecurityToken(
        issuer: _settings.Issuer,
        audience: _settings.Audience,
        claims: claims,
        expires: DateTime.UtcNow.AddDays(7),
        signingCredentials: creds
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

2.2 Token Structure

Part	Content
Header	`{ "alg": "HS256", "typ": "JWT" }`
Payload	`{ "nameid": "<userId>", "email": "<email>", "role": "<role>", "exp": <timestamp>, "iss": "SmartMedicationDispenser", "aud": "SmartMedicationDispenser" }`
Signature	HMAC-SHA256 using secret key

2.3 Token Claims

Claim	Type	Description	Example
`nameid`	ClaimTypes.NameIdentifier	User ID (GUID)	`a1b2c3d4-...`
`email`	ClaimTypes.Email	User email	`patient@demo.com`
`role`	ClaimTypes.Role	User role	`Patient`, `Caregiver`, `Admin`
`exp`	Standard	Expiration (7 days)	Unix timestamp
`iss`	Standard	Issuer	`SmartMedicationDispenser`
`aud`	Standard	Audience	`SmartMedicationDispenser`

2.4 Token Validation

Configured in `Program.cs`:

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)),
            ValidateIssuer = true,
            ValidIssuer = "SmartMedicationDispenser",
            ValidateAudience = true,
            ValidAudience = "SmartMedicationDispenser",
            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero  // No clock skew tolerance
        };
    });
```

Validation checks:

1. Signing key is valid (HS256)
2. Issuer matches configuration
3. Audience matches configuration
4. Token has not expired
5. No clock skew tolerance (strict expiry)

2.5 Using JWT in Requests

Web Portal (Axios + localStorage):

> **Note:** The web portal currently uses a mock AuthContext with `mockUsers`. The Axios client (`api/client.ts`) is fully implemented for real API calls.

```
// api/client.ts — reads from localStorage key "token"
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

AuthContext (mock) stores under different keys:
- `localStorage.jwt_token` — mock token string
- `localStorage.user_data` — JSON user object

Mobile App (AsyncStorage + In-Memory):

```
// Token persisted in AsyncStorage under key "token"
await AsyncStorage.setItem('token', data.token);

// Also set in-memory for Axios interceptor
export let authToken: string | null = null;
export function setAuthToken(token: string | null) { authToken = token; }

// Request interceptor adds Bearer token from in-memory variable
api.interceptors.request.use((config) => {
  if (authToken) config.headers.Authorization = `Bearer ${authToken}`;
  return config;
});

// On app launch, token is restored from AsyncStorage → setAuthToken()
```

cURL Example:

```
curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..." \
     http://localhost:5000/api/devices
```

---

3. Password Hashing

3.1 BCrypt Implementation

```
// JwtAuthService.cs
public (string Hash, string Salt)? HashPassword(string password)
{
    var salt = BCryptNet.GenerateSalt();
    var hash = BCryptNet.HashPassword(password, salt);
    return (hash, salt);
}

public bool VerifyPassword(string password, string hash) =>
    BCryptNet.Verify(password, hash);
```

3.2 Security Properties

Property	Value
Algorithm	BCrypt
Salt	Auto-generated per password
Work factor	Default (10 rounds = 1024 iterations)
Storage	Only hash stored in database (`User.PasswordHash`)
Verification	Constant-time comparison (BCrypt.Verify)

---

4. API Key Authentication (X-API-Key)

4.1 Overview

API keys are used for machine-to-machine authentication — ESP32 devices and external systems calling webhook/sync endpoints.

4.2 API Key Lifecycle

```
1. User creates API key in Web Portal (Integrations page)
   → POST /api/integrations/devices/{deviceId}/api-keys
   → Returns: { apiKeyId, plainKey: "sk_..." }

2. Plain key is shown ONCE (user must copy it)

3. SHA256 hash of key is stored in database (DeviceApiKeys table)

4. Device/system uses key in X-API-Key header:
   → X-API-Key: sk_...

5. DeviceApiKeyResolver:
   → Computes SHA256(provided_key)
   → Looks up hash in DeviceApiKeys table
   → Returns associated DeviceId
   → Updates LastUsedAtUtc
```

4.3 API Key Resolution

```
// DeviceApiKeyResolver.cs
public async Task<Guid?> ResolveDeviceIdFromApiKeyAsync(string? apiKey, CancellationToken ct)
{
    if (string.IsNullOrWhiteSpace(apiKey)) return null;

    // Compute SHA256 hash of provided key
    var hash = ComputeSha256(apiKey);

    // Look up hash in database
    var key = await _repository.GetByHashAsync(hash, ct);
    if (key == null) return null;

    // Update last used timestamp
    key.LastUsedAtUtc = DateTime.UtcNow;
    await _unitOfWork.SaveChangesAsync(ct);

    return key.DeviceId;
}
```

4.4 API Key Format

Property	Value
Prefix	`sk_`

Property	Value
Key length	32+ characters
Storage	SHA256 hash only (64-char hex)
Scope	One key per device
Revocation	Delete via API or web portal

4.5 Endpoints Using X-API-Key

Endpoint	Description
`POST /api/webhooks/incoming`	Incoming webhook from cloud/hardware
`POST /api/integrations/sync`	Bulk sync data from cloud
`POST /api/v1/devices/{id}/heartbeat`	Device heartbeat (fallback auth)
`POST /api/v1/events`	Device events (fallback auth)

---

5. User Roles & Authorization

5.1 Role Definitions

Role	Value	Description	Access
Patient	0	End user who takes medication	Own devices, doses, history
Caregiver	1	Monitors assigned patients	Patient's devices + own dashboard
Admin	2	Platform administrator	Full system access

5.2 Authorization Model

The current MVP uses **resource-based authorization**:

```
// All controllers extract userId from JWT claims
private Guid? UserId => Guid.TryParse(
    User.FindFirstValue(ClaimTypes.NameIdentifier), out var id) ? id : null;

// Handlers verify ownership before returning/modifying data
// Example: GetDevicesQueryHandler
var devices = await _deviceRepository.GetByUserIdAsync(query.UserId, ct);
// Returns only devices owned by the requesting user
```

5.3 Authorization Rules

Resource	Patient	Caregiver	Admin
Own devices	Read, Create, Pause, Resume	Via patient assignment	All
Own containers	Full CRUD	Via patient assignment	All
Own schedules	Full CRUD	Via patient assignment	All
Dispense events	Dispense, Confirm, Delay	View (via patient)	All
History	Own device events	Patient's device events	All
Notifications	Own notifications	Own + patient alerts	All
Travel sessions	Start/End own	View patient's	All
Webhooks	Own endpoints	Own endpoints	All
API keys	Own device keys	Own device keys	All

5.4 Controller-Level Authorization

```
// [Authorize] — Requires any valid JWT
[ApiController]
[Route("api/[controller]")]
[Authorize]
public class DevicesController : ControllerBase { ... }

// [AllowAnonymous] — No auth required
[HttpPost("register")]
[AllowAnonymous]
public async Task<ActionResult> Register(...) { ... }

// No [Authorize] at controller level, mixed per endpoint
[ApiController]
[Route("api/integrations")]
public class IntegrationsController : ControllerBase
{
    [Authorize]  // JWT for webhook management
    [HttpGet("webhooks")]
    public async Task<ActionResult> GetWebhooks(...) { ... }

    // No [Authorize] — uses X-API-Key instead
    [HttpPost("sync")]
    public async Task<ActionResult> Sync(...) { ... }
}
```

6. Security Configuration

6.1 JWT Configuration (appsettings.json)

```
{
  "Jwt": {
    "SecretKey": "SmartMedicationDispenser_MVP_SecretKey_AtLeast32Characters!",
    "Issuer": "SmartMedicationDispenser",
    "Audience": "SmartMedicationDispenser"
  }
}
```

6.2 Production Security Checklist

Item	Status	Action
JWT secret key	⚠️ Change for production	Use 64+ char random key from env variable
HTTPS	⚠️ Not enforced in MVP	Enable TLS via reverse proxy
CORS	⚠️ AllowAny in MVP	Restrict to specific frontend domains
Rate limiting	❌ Not implemented	Add per-IP and per-user rate limits
Password policy	✅ FluentValidation	Min length enforced on register
Token expiry	✅ 7 days	Consider shorter for production
Clock skew	✅ Zero	Strict token expiry validation
BCrypt hashing	✅ Implemented	Passwords never stored in plain text
API key hashing	✅ SHA256	Plain keys never stored

6.3 CORS Configuration (Current — MVP)

```
// Program.cs — allows all origins for development
app.UseCors(policy =>
{
    policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader();
});
```

Production recommendation:

```
app.UseCors(policy =>
{
    policy.WithOrigins("https://app.smartdispenser.ch")
          .AllowAnyMethod()
          .AllowAnyHeader()
          .AllowCredentials();
});
```

---

7. Authentication Flows

7.1 Login Flow

```
Client                              Server
  |                                   |
  | POST /api/auth/login              |
  | { email, password }               |
  |──────────────────────────────────▶|
  |                                   |
  |                                   | 1. Find user by email
  |                                   | 2. BCrypt.Verify(password, hash)
  |                                   | 3. Generate JWT (7 days)
  |                                   |
  | 200 OK                            |
  | { token, email, fullName, role }  |
  |◀──────────────────────────────────|
  |                                   |
  | Store token (localStorage/memory) |
  |                                   |
```

7.2 Protected Request Flow

```
Client                              Server
  |                                   |
  | GET /api/devices                  |
  | Authorization: Bearer <jwt>       |
  |──────────────────────────────────▶|
  |                                   |
  |                 JWT Middleware:    |
  |                 1. Extract token   |
  |                 2. Validate sig    |
  |                 3. Check expiry    |
  |                 4. Set claims      |
  |                                   |
  |                 Controller:        |
  |                 5. Extract userId  |
  |                 6. Query user's    |
  |                    devices only    |
  |                                   |
  | 200 OK                            |
  | [{ device1 }, { device2 }]        |
  |◀──────────────────────────────────|
  |                                   |
```

7.3 API Key Flow (Device/Cloud)

```
Device                              Server
  |                                   |
  | POST /api/webhooks/incoming       |
  | X-API-Key: sk_abc123...           |
  | { eventType, data }               |
  |──────────────────────────────────▶|
  |                                   |
  |           DeviceApiKeyResolver:   |
  |           1. SHA256(sk_abc123...) |
  |           2. Lookup hash in DB    |
  |           3. Return deviceId      |
  |           4. Update lastUsed      |
  |                                   |
  |           Handler:                |
  |           5. Process event for    |
  |              resolved device      |
  |                                   |
  | 202 Accepted                      |
  |◀──────────────────────────────────|
  |                                   |
```

7.4 401 Handling (Auto-Logout)

Web Portal:

```
api.interceptors.response.use(
  (r) => r,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);
```

Mobile App:

```
api.interceptors.response.use(
  (r) => r,
  (err) => {
    if (err.response?.status === 401) authToken = null;
    return Promise.reject(err);
  }
);
```

Integrations & Webhooks

Smart Medication Dispenser — Outgoing/Incoming Webhooks, Cloud Sync & Device API Keys

---

1. Integration Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        INTEGRATION ARCHITECTURE                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  INCOMING (to our API)                    OUTGOING (from our API)        │
│                                                                          │
│  ┌──────────────┐                        ┌──────────────┐               │
│  │   ESP32      │  POST /api/v1/events   │  Your Server │               │
│  │   Firmware   │──────────────────────▶ │  POST https: │               │
│  │              │  Device JWT / X-Key    │  //callback  │               │
│  └──────────────┘                        └──────┬───────┘               │
│                                                  ▲                       │
│  ┌──────────────┐                                │                       │
│  │   External   │  POST /api/webhooks/   ┌──────┴───────┐               │
│  │   Cloud      │  incoming              │  Webhook     │               │
│  │   System     │──────────────────────▶ │  Delivery    │               │
│  └──────────────┘  X-API-Key             │  Service     │               │
│                                          └──────────────┘               │
│  ┌──────────────┐                                ▲                       │
│  │   External   │  POST /api/integrations/       │                       │
│  │   Cloud      │  sync                  ┌──────┴───────┐               │
│  │   System     │──────────────────────▶ │  Background  │               │
│  └──────────────┘  X-API-Key             │  Job:        │               │
│                                          │  Missed Dose │               │
│                                          │  Low Stock   │               │
│                                          └──────────────┘               │
│                                                                          │
│  TRIGGERS FOR OUTGOING WEBHOOKS:                                         │
│  • dispense.confirmed (user confirmed dose intake)                       │
│  • notification.missed_dose (dose marked as missed)                      │
│  • notification.low_stock (container below threshold)                    │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

---

2. Outgoing Webhooks

2.1 Overview

Outgoing webhooks notify external systems when events occur in the platform. Users configure webhook URLs in the web portal under **Integrations**.

2.2 Creating a Webhook

Web Portal: Integrations → Outgoing Webhooks → Add Webhook

API:

```
POST /api/integrations/webhooks
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "url": "https://your-server.com/webhook/dispenser",
  "secret": "optional-signing-secret",
  "description": "My notification endpoint"
}
```

Response:

```
{
  "id": "wh1a2b3c...",
  "url": "https://your-server.com/webhook/dispenser",
  "isActive": true,
  "description": "My notification endpoint",
  "lastTriggeredAtUtc": null,
  "lastStatus": null,
  "createdAtUtc": "2026-02-10T12:00:00Z"
}
```

2.3 Webhook Event Types

Event Type	Trigger	Data Included
`dispense.confirmed`	User confirmed taking a dose	dispenseEventId, deviceId, medicationName
`notification.missed_dose`	Dose marked as missed (60 min timeout)	dispenseEventId, deviceId
`notification.low_stock`	Container below low stock threshold	containerId, deviceId, medicationName, quantity

2.4 Webhook Payload Format

```
{
  "eventType": "dispense.confirmed",
  "timestampUtc": "2026-02-10T08:05:00Z",
  "data": {
    "dispenseEventId": "e1f2g3...",
    "deviceId": "d1e2f3...",
    "medicationName": "Metformin 500mg"
  }
}
```

2.5 Webhook Signature Verification

If a **secret** is configured, the webhook includes an HMAC-SHA256 signature:

Header: `X-Webhook-Signature: sha256=<hex-encoded-hmac>`

Verification (server-side):

```
import hmac
import hashlib

def verify_webhook(payload_bytes, signature_header, secret):
    expected = hmac.new(
        secret.encode(), payload_bytes, hashlib.sha256
    ).hexdigest()
    provided = signature_header.replace("sha256=", "")
    return hmac.compare_digest(expected, provided)
```

```
const crypto = require('crypto');

function verifyWebhook(body, signatureHeader, secret) {
  const expected = crypto
    .createHmac('sha256', secret)
    .update(body)
    .digest('hex');
  const provided = signatureHeader.replace('sha256=', '');
  return crypto.timingSafeEqual(
    Buffer.from(expected), Buffer.from(provided)
  );
}
```

2.6 Webhook Delivery Details

Property	Value
HTTP Method	POST
Content-Type	application/json
Timeout	10 seconds
Retry	None (MVP — fire and forget)
Headers	`X-Webhook-Event: dispenser.event`, `User-Agent: SmartMedicationDispenser/1.0`, `X-Webhook-Signature: sha256=<hmac>` (if secret set)

2.7 Managing Webhooks

Action	Method	Endpoint
List all	GET	`/api/integrations/webhooks`

Action	Method	Endpoint
Create	POST	`/api/integrations/webhooks`
Delete	DELETE	`/api/integrations/webhooks/{id}`

---

3. Incoming Webhooks

3.1 Overview

External systems (cloud platforms, device gateways) can push events to the API using the incoming webhook endpoint.

3.2 Endpoint

```
POST /api/webhooks/incoming
X-API-Key: <device-api-key>
Content-Type: application/json
```

3.3 Request Format

```
{
  "eventType": "heartbeat",
  "deviceId": "optional-override-guid",
  "data": {}
}
```

3.4 Supported Event Types

eventType	Description	Data Fields
`heartbeat`	Update device last seen time	—
`device_status`	Set device active/paused	`{ "status": "Active" | "Paused" }`
`dispense_completed`	Record a dispense event	`{ "containerId": "<guid>", "scheduleId": "<guid>" }`
`low_stock`	Create low-stock notification	`{ "containerId": "<guid>", "message": "Optional text" }`

3.5 Examples

Heartbeat:

```
curl -X POST http://localhost:5000/api/webhooks/incoming \
  -H "Content-Type: application/json" \
  -H "X-API-Key: sk_your_key_here" \
  -d '{"eventType":"heartbeat","data":{}}'
```

Dispense Completed:

```
curl -X POST http://localhost:5000/api/webhooks/incoming \
  -H "Content-Type: application/json" \
  -H "X-API-Key: sk_your_key_here" \
  -d '{
    "eventType": "dispense_completed",
    "data": {
      "containerId": "c1d2e3-...",
      "scheduleId": "s1a2b3-..."
    }
  }'
```

Low Stock:

```
curl -X POST http://localhost:5000/api/webhooks/incoming \
  -H "Content-Type: application/json" \
  -H "X-API-Key: sk_your_key_here" \
  -d '{
    "eventType": "low_stock",
    "data": {
      "containerId": "c1d2e3-...",
      "message": "Metformin: 5 pills remaining"
    }
  }'
```

3.6 Authentication

The incoming webhook resolves the device from the `X-API-Key` header:

1. Extract `X-API-Key` from request headers
2. Compute SHA256 hash of the key
3. Look up the hash in `DeviceApiKeys` table
4. Return the associated `DeviceId`
5. If `deviceId` is provided in the body, it must match the key's device

Error responses:

- `401 Unauthorized` — Missing or invalid API key
- `403 Forbidden` — Body deviceId doesn't match key's device
- `400 Bad Request` — Unknown event type

---

4. Cloud Sync

4.1 Overview

The sync endpoint allows external cloud systems to push bulk data (device status + dispense events) to the platform.

4.2 Endpoint

```
POST /api/integrations/sync
X-API-Key: <device-api-key>
Content-Type: application/json
```

4.3 Request Format

```
{
  "deviceId": "<device-guid-matching-the-api-key>",
  "device": {
    "status": "Active",
    "timeZoneId": "America/New_York",
    "lastHeartbeatAtUtc": "2026-02-10T12:00:00Z"
  },
  "events": [
    {
      "containerId": "<guid>",
      "scheduleId": "<guid>",
      "scheduledAtUtc": "2026-02-10T08:00:00Z",
      "status": "Confirmed",
      "dispensedAtUtc": "2026-02-10T08:01:00Z",
      "confirmedAtUtc": "2026-02-10T08:05:00Z"
    }
  ]
}
```

4.4 Sync Rules

Rule	Description
`deviceId` must match API key	The device ID in the body must be the device associated with the API key
`device` is optional	Only include if updating device status/metadata
`events` is optional	Only include if pushing dispense events
Idempotent	Sync can be called repeatedly; events are created if not already existing
Status values	`Pending`, `Dispensed`, `Confirmed`, `Missed`, `Delayed`

4.5 Example

```
curl -X POST http://localhost:5000/api/integrations/sync \
  -H "Content-Type: application/json" \
  -H "X-API-Key: sk_your_key_here" \
  -d '{
    "deviceId": "d1e2f3-...",
    "device": {
      "status": "Active",
      "lastHeartbeatAtUtc": "2026-02-10T12:00:00Z"
    },
    "events": [
      {
        "containerId": "c1d2e3-...",
        "scheduleId": "s1a2b3-...",
        "scheduledAtUtc": "2026-02-10T08:00:00Z",
        "status": "Confirmed",
        "dispensedAtUtc": "2026-02-10T08:01:00Z",
        "confirmedAtUtc": "2026-02-10T08:05:00Z"
      }
    ]
  }'
```

---

5. Device API Keys

5.1 Creating API Keys

Web Portal: Integrations → Device API Keys → Select device → Create Key

API:

```
POST /api/integrations/devices/{deviceId}/api-keys
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "name": "Production ESP32"
}
```

Response:

```
{
  "apiKeyId": "k1a2b3...",
  "plainKey": "sk_8f4a2b1c9e3d7f6a5b4c3d2e1f0a9b8c"
}
```

Important: The `plainKey` is shown **only once**. Store it securely.

5.2 API Key Storage

What's Stored	Where	Format
SHA256 hash of key	`DeviceApiKeys.KeyHash`	64-character hex string
Key metadata	`DeviceApiKeys` table	Name, device ID, timestamps
Plain key	**NOT stored**	Shown once on creation

5.3 Managing API Keys

Action	Method	Endpoint	Auth
List keys for device	GET	`/api/integrations/devices/{deviceId}/api-keys`	JWT
Create key	POST	`/api/integrations/devices/{deviceId}/api-keys`	JWT
Revoke key	DELETE	`/api/integrations/devices/{deviceId}/api-keys/{apiKeyId}`	JWT

5.4 API Key Usage

Include the key in the `X-API-Key` header:

```
curl -X POST http://localhost:5000/api/webhooks/incoming \
  -H "X-API-Key: sk_8f4a2b1c9e3d7f6a5b4c3d2e1f0a9b8c" \
  -H "Content-Type: application/json" \
  -d '{"eventType":"heartbeat","data":{}}'
```

---

6. Device API (ESP32 → Cloud)

6.1 Overview

The Device API (`/api/v1/*`) is the primary interface for ESP32 firmware to communicate with the cloud backend.

6.2 Authentication

Devices authenticate using either:
- **Device JWT** — Obtained during device registration (`Authorization: Bearer <token>`)
- **X-API-Key** — Created via the web portal (`X-API-Key: sk_...`)

6.3 Device Registration Flow

```
ESP32 (First Boot)                    Server
  |                                     |
  | POST /api/v1/devices/register       |
  | { deviceId, deviceType,             |
  |   firmwareVersion, macAddress }     |
  |──────────────────────────────────────▶|
  |                                     |
  |                                     | Create device record
  |                                     | Generate device token
  |                                     |
  | { success, deviceToken,             |
  |   apiEndpoint, heartbeatInterval }  |
  |◀──────────────────────────────────────|
  |                                     |
  | Store deviceToken in NVS            |
  | Begin heartbeat loop (60s)          |
  |                                     |
```

6.4 Heartbeat Flow

```
ESP32 (Every 60 seconds)              Server
  |                                     |
  | POST /api/v1/devices/{id}/heartbeat |
  | Authorization: Bearer <token>       |
  | { battery, wifiSignal, slots,       |
  |   temperature, humidity, errors }   |
  |──────────────────────────────────────▶|
  |                                     |
  |                                     | Update device status
  |                                     | Check for commands
  |                                     |
  | { success, server_time,             |
  |   commands: [], next_heartbeat }    |
  |◀──────────────────────────────────────|
  |                                     |
```

6.5 Event Reporting Flow

```
ESP32 (On Event)                      Server
  |                                     |
  | POST /api/v1/events                 |
  | { deviceId, event: "DOSE_TAKEN",    |
  |   timestamp, data: { slot, pills }} |
  |──────────────────────────────────────▶|
  |                                     |
  |                                     | Log event
  |                                     | Update dispense status
  |                                     | Trigger notifications
  |                                     | Send outgoing webhooks
  |                                     |
  | { success, event_id }               |
  |◀──────────────────────────────────────|
  |                                     |
```

7. Background Job: Missed Dose & Low Stock

7.1 Overview

A hosted background service runs every **5 minutes** to detect:

1. **Missed doses** — Dispense events older than 60 minutes still in Pending/Dispensed status
2. **Low stock** — Containers where quantity is below the low stock threshold

7.2 Missed Dose Detection

```
Every 5 minutes:
  1. Query DispenseEvents WHERE Status IN (Pending, Dispensed)
     AND ScheduledAtUtc < (UtcNow - 60 minutes)

  2. For each overdue event:
     a. Set Status = Missed, MissedMarkedAtUtc = UtcNow
     b. Create Notification for patient ("Missed dose")
     c. If patient has caregiver → Create Notification for caregiver
     d. Send outgoing webhooks (notification.missed_dose)
```

7.3 Low Stock Detection

```
Every 5 minutes:
  1. Query Containers WHERE Quantity < LowStockThreshold

  2. For each low-stock container:
     a. Check if unread LowStock notification already exists
     b. If no existing notification:
        - Create Notification ("Low stock: {med} has {qty} remaining")
        - Send outgoing webhooks (notification.low_stock)
```

7.4 Webhook Delivery in Background Job

When the background job creates notifications, it also delivers outgoing webhooks:

```
// For each active webhook endpoint of the user:
var payload = new { eventType, timestampUtc = DateTime.UtcNow, data };
var status = await webhookDelivery.SendAsync(ep.Url, payload, ep.Secret, ct);
await webhookEndpoints.UpdateLastTriggeredAsync(ep.Id, status.ToString(), ct);
```

8. Integration Quick Start

8.1 Step-by-Step Setup

```
1. Login to web portal (http://localhost:5173)
   → Use demo credentials: patient@demo.com / Demo123!

2. Navigate to Integrations page

3. Create a Device API Key:
   → Select a device (e.g., "Home Dispenser")
   → Click "Create API Key"
   → Copy the key (sk_...) — shown only once!

4. (Optional) Create an Outgoing Webhook:
   → Enter your callback URL
   → (Optional) Set a signing secret
   → Click "Create Webhook"

5. Test with incoming webhook:
   curl -X POST http://localhost:5000/api/webhooks/incoming \
     -H "Content-Type: application/json" \
     -H "X-API-Key: sk_your_copied_key" \
     -d '{"eventType":"heartbeat","data":{}}'

   Expected: 202 Accepted

6. Test with sync:
   curl -X POST http://localhost:5000/api/integrations/sync \
     -H "Content-Type: application/json" \
     -H "X-API-Key: sk_your_copied_key" \
     -d '{
       "deviceId": "<device-guid-from-web-portal>",
       "device": {"status":"Active"},
       "events": []
     }'

   Expected: 202 Accepted
```

8.2 Integration Checklist

Step	Action	Status
1	Create device API key in web portal	☐
2	Store API key securely in your system	☐
3	Implement heartbeat calls (every 60s)	☐
4	Implement event reporting (dose/error/battery)	☐
5	(Optional) Set up outgoing webhook endpoint	☐
6	(Optional) Implement webhook signature verification	☐
7	(Optional) Implement sync for bulk updates	☐

Step	Action	Status
8	Test end-to-end with demo data	☐

---

9. Error Handling

9.1 Incoming Webhook Errors

HTTP Status	Cause	Resolution
401	Missing or invalid X-API-Key	Check key is correct and not revoked
403	Body deviceId doesn't match key	Use correct deviceId or omit it
400	Unknown eventType or invalid data	Check supported event types
500	Server error	Check server logs

9.2 Outgoing Webhook Errors

Issue	Behavior	Resolution
Target URL unreachable	Logged as warning, delivery skipped	Check your server URL
Timeout (>10s)	Request cancelled	Optimize your webhook handler
Non-2xx response	Status recorded in webhook endpoint	Check your server logs
Connection refused	Logged as warning	Ensure server is running

9.3 Sync Errors

HTTP Status	Cause	Resolution
401	Invalid API key	Check key
400	DeviceId mismatch or invalid data	Ensure deviceId matches API key's device
400	Invalid event status value	Use: Pending, Dispensed, Confirmed, Missed, Delayed